

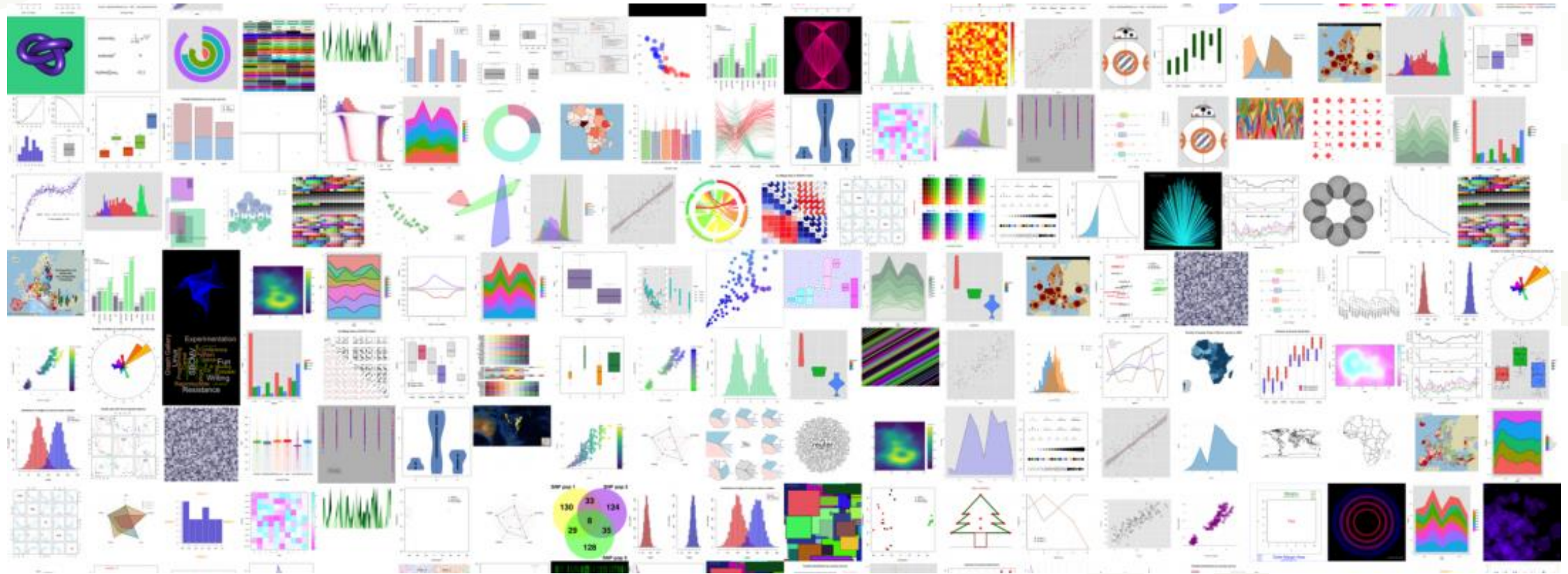


R计算基础和统计绘图

陈同

中国中医科学院

生物信息为什么使用 R? 统计、绘图神器



- 统计检验、数据整理和绘图可以写在同一个分析脚本中
- CRAN 提供通用统计工具, Bioconductor 面向组学数据
- 代码、参数和结果可以共同保存, 便于复查与复现

R的软件生态

基础环境

R 解释器负责计算

RStudio 或 Positron 负责编辑与调试

R Markdown 或 Quarto 组织代码和报告

扩展资源

CRAN: 通用统计与绘图包

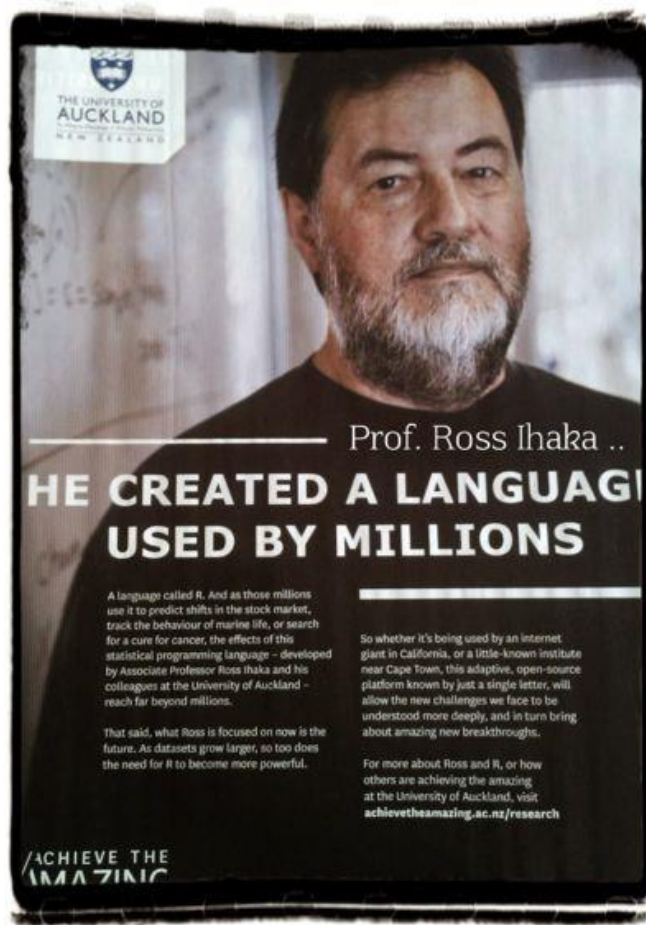
Bioconductor: 基因组和组学分析包

GitHub: 开发版软件与协作项目

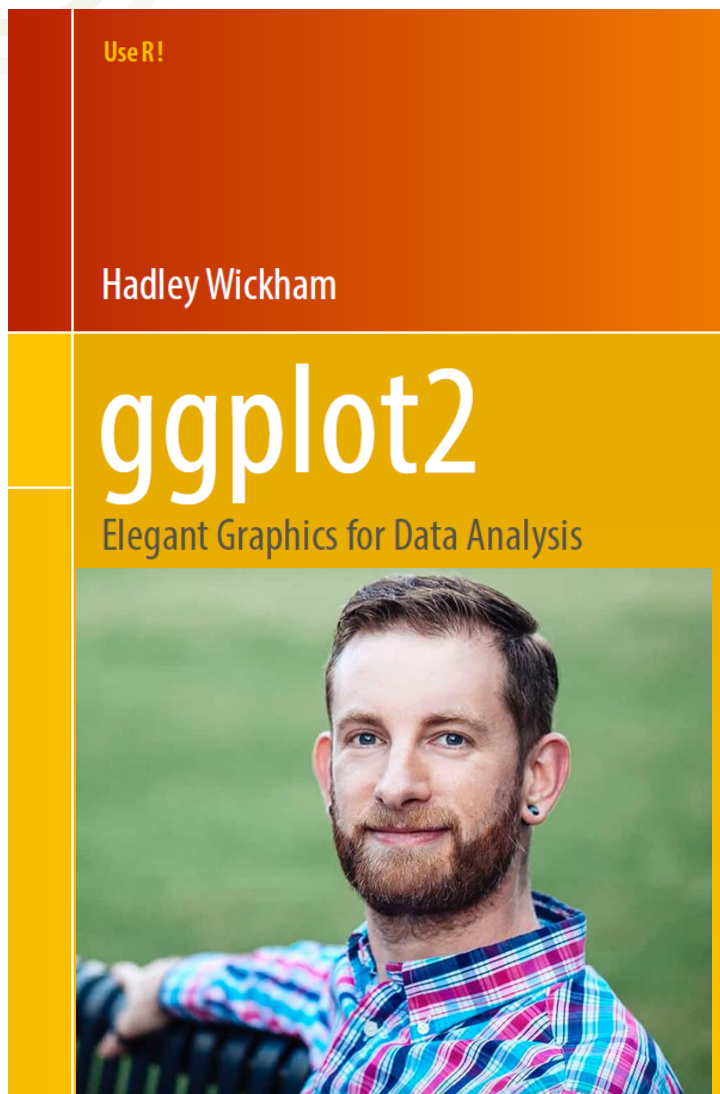
同一段 R 代码可以在 Windows、macOS、Linux 和服务端中运行

R语言谁创造的?

- R是一种有强大统计分析和作图功能的编程语言，受S语言和Scheme影响
- 由新西兰奥克兰大学统计系的Ross Ihaka和Robert Gentleman两人合作编写
- 由R开发核心小组负责维护，更新速度极快
- 截止目前R已更新至4.6.1版本



R语言学习书籍



- R for Data Science: 数据导入、整理、可视化与 workflows
- ggplot2: Elegant Graphics for Data Analysis: 图形语法与绘图
- RStudio Cheatsheets: 常用函数的快速索引
- 函数帮助与 vignette: 优先查当前安装版本的文档

在 R 中直接查文档

```
?read.table  
help.search("heatmap")  
vignette(package = "ggplot2")
```

本节学习目标

- 理解向量、因子、矩阵、列表和数据框的差别
- 正确使用 `[`、`[[`、`$` 与 `drop = FALSE` 提取数据
- 读入并检查表达矩阵、样本信息和差异分析结果
- 完成基本统计、`ggplot2` 绘图和热图绘制
- 借助 `AI` 解释和调试代码，同时独立核对对象与结果

R开发环境部署

个人电脑

先从 CRAN 安装 R

再安装 RStudio Desktop

安装后运行 `R.version.string`

服务器

优先使用管理员提供的 R 环境

也可使用 conda、容器或源码安装

确认 R 与包库路径一致

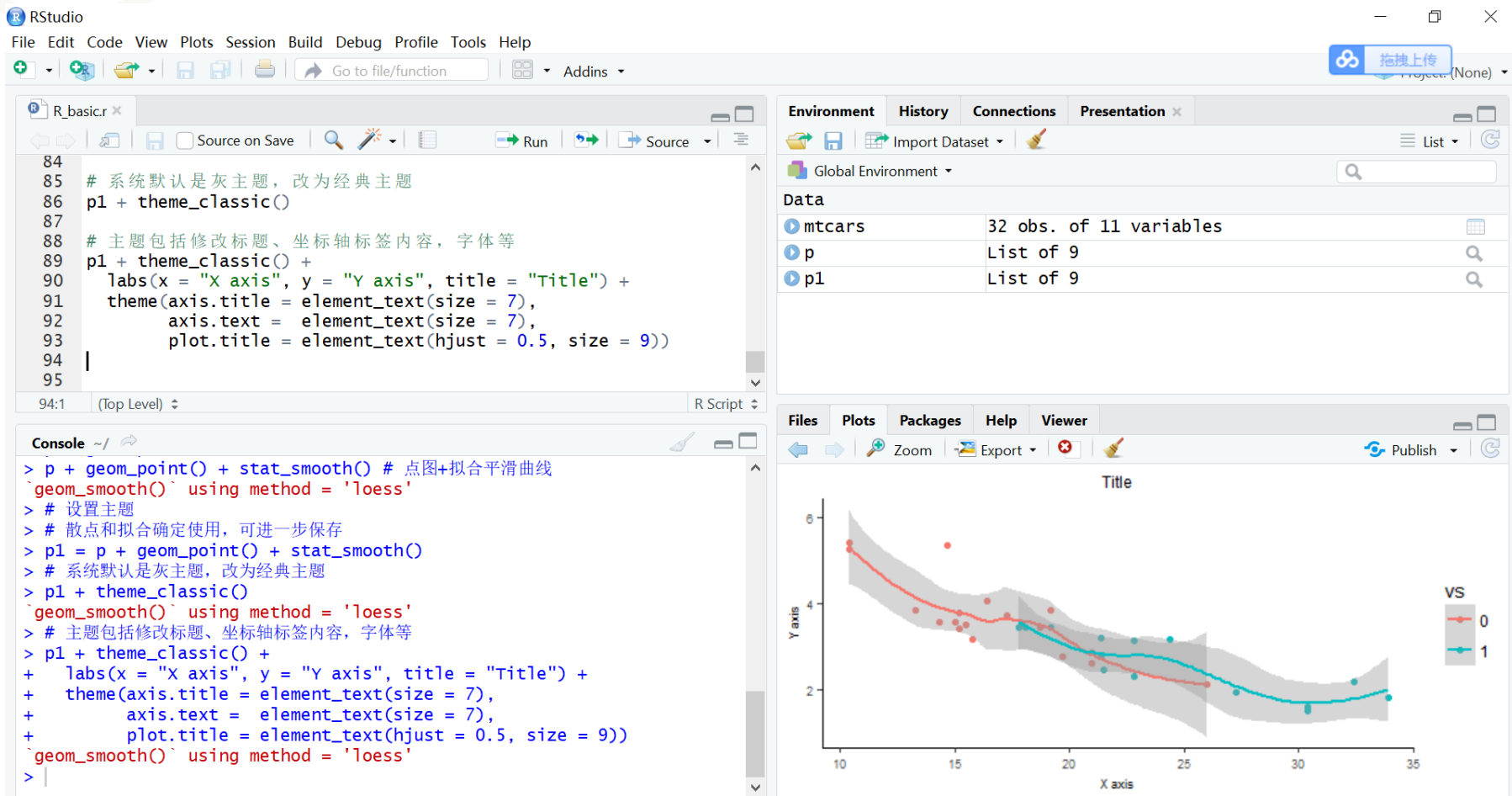
安装后核对

```
R.version.string
```

```
R.home()
```

```
.libPaths()
```

Rstudio的4个主要区域



脚本编辑区

编写并保存代码

Console

查看执行和报错

Environment / History

对象和命令历史

Files / Plots / Help
文件、结果和帮助

R项目目录和文件路径

推荐方式

为每次分析建立 RStudio Project
数据放在 `data/`，结果放在 `results/`
脚本只使用相对路径

避免的问题

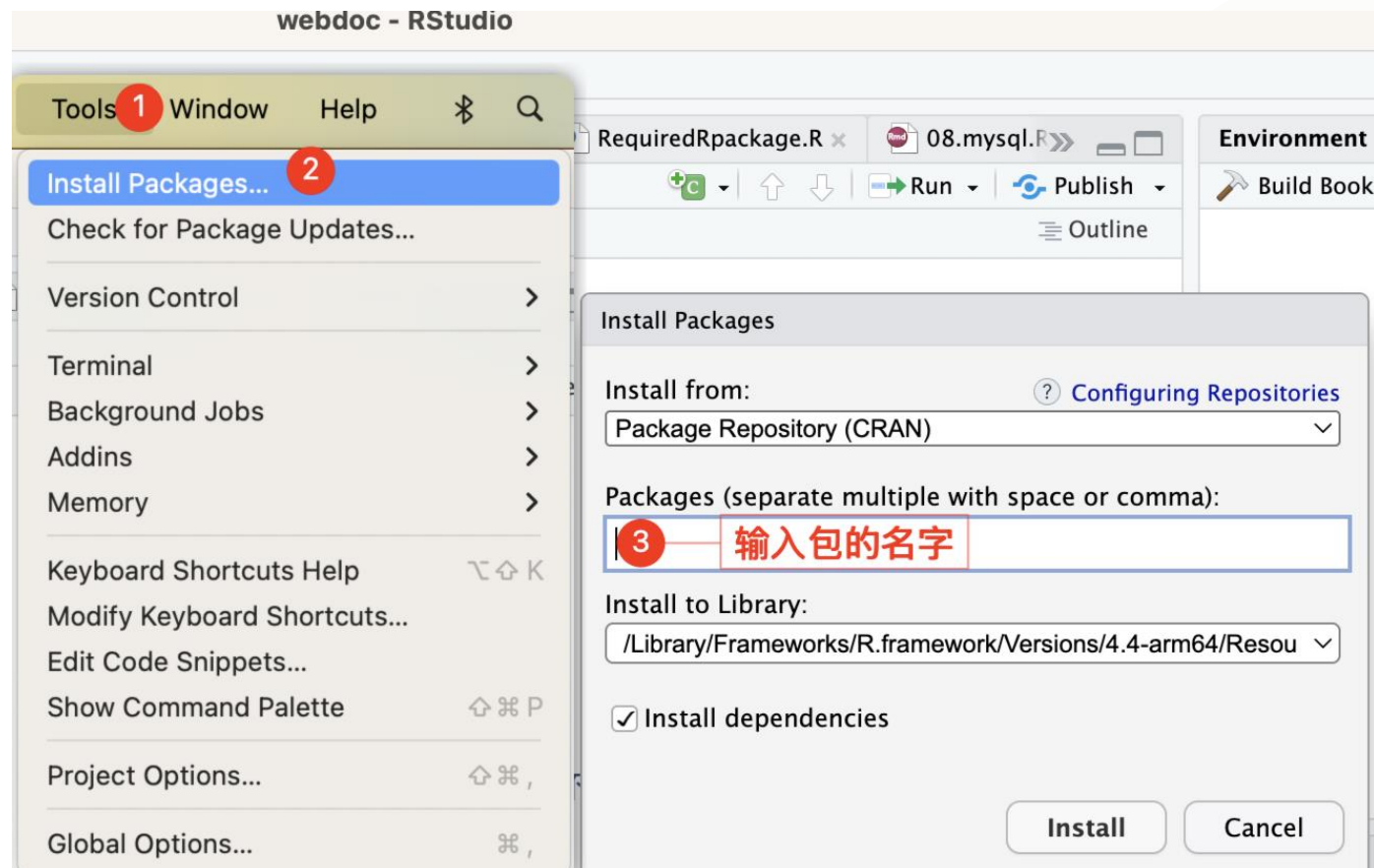
不要依赖临时工作目录
不要在脚本中写个人电脑绝对路径
不要把输入数据覆盖为结果文件

先确认位置

```
getwd()  
list.files()  
here::here("data", "counts.tsv")
```

R语言包安装

- 直接在 Rstudio 界面安装 CRAN 包活其它下载到本地的包



R包的命令行安装和加载

CRAN 与 GitHub

```
install.packages("ggplot2")  
remotes::install_github("Tong-  
Chen/ImageGP")
```

安装通常只做一次

Bioconductor

```
if (!requireNamespace("BiocManager"))  
  install.packages("BiocManager")  
BiocManager::install("DESeq2")
```

加载在每次新会话中执行

包加载和版本检查

```
library(ggplot2)  
packageVersion("ggplot2")  
BiocManager::valid()
```

R语言包安装

```
site = "https://mirrors.tuna.tsinghua.edu.cn/CRAN"

# options(BioC_mirror =
"https://mirrors.tuna.tsinghua.edu.cn/bioconductor")

options(BioC_mirror="https://mirrors.westlake.edu.cn/bioconductor")

if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager", repos = site)

a = rownames(installed.packages())

install_bioc <- c(
  "ape",
  "agricolae",
  "sva",
  "tidyr",
  "tidyverse",
  "tximport",
  "vegan")
```

```
for (i in install_bioc) {
  if (!i %in% a)
    BiocManager::install(i)
  a = rownames(installed.packages())
}

# 安装 git 上的包
# remotes::install_github("Tong-Chen/ImageGP")
# remotes::install_git("https://gitee.com/ct5869/ImageGP.git")
```

R可重复运行环境

- `set.seed()` 固定含随机过程的结果
- `sessionInfo()` 记录 R、操作系统和已加载包版本
- `renv::snapshot()` 保存项目依赖, `renv::restore()` 恢复环境
- 输入文件、脚本、参数和输出使用清晰目录保存

最小记录

```
set.seed(101)
renv::init()
renv::snapshot()
sessionInfo()
```

R对象与赋值

对象名指向内存中的值

```
count <- 10  
gene <- "SOX2"  
is_de <- TRUE  
  
ls()  
rm(count)
```

- 使用 `<-` 赋值，使用有含义的对象名
- R 区分大小写，`gene` 与 `Gene` 是两个对象
- 覆盖对象前先确认新值的类型和维度

R数据基本类型与特殊值

基本类型

numeric: 1、2.5	数值型
integer: 2L	整型
character: "SOX2"	字符型
logical: TRUE、FALSE	逻辑型

特殊值

NA: 缺失值

NaN: 无法定义的数值

Inf: 无穷大

NULL: 对象或成分不存在

课堂代码使用 TRUE 和 FALSE，避免用可被重新赋值的 T 和 F

R向量与索引

向量保存同一种基本类型

```
x <- c(5, 8, 3, 8, 10)
x[2]
x[c(1, 5)]
x[x >= 8]
x[-1]
length(x)
```

```
> x <- c(5, 8, 3, 8, 10)
> x[2]
[1] 8
> x[c(1, 5)]
[1] 5 10
> x[x >= 8]
[1] 8 8 10
> x[-1]
[1] 8 3 8 10
> length(x)
[1] 5
```

- 索引从 1 开始
- 逻辑条件返回与向量等长的
TRUE / FALSE
- 混合类型会发生自动转换，先
用 `typeof()` 检查

R向量命名

向量保存同一种基本类型

```
x <- c(5, 8, 3, 8, 10)
names(x) <- letters[1:5]
x[1]
x['b']
as.vector(x[1])
```

```
> x <- c(5, 8, 3, 8, 10)
> names(x) <- letters[1:5]
> x[1]
a
5
> x['b']
b
8
> as.vector(x[1])
[1] 5
```

R字符向量的因子类型

levels 决定顺序和参照水平

```
group <- factor(  
  c("trt", "untrt", "trt"),  
  levels = c("untrt", "trt")  
)  
levels(group)
```

```
group <- factor(  
  c("trt", "untrt", "trt"),  
  levels = c("untrt", "trt"),  
  ordered=TRUE  
)  
group
```

- 因子的核心作用是表示有固定水平的分类变量
- 绘图顺序和统计模型参照水平都受 levels 影响
- 字符转因子前先检查拼写、空格和缺失值

R字符向量的因子类型

levels 决定顺序和参照水平

```
group <- factor(  
  c("trt", "untrt", "trt"),  
  levels = c("untrt", "trt")  
)  
levels(group)
```

```
group <- factor(  
  c("trt", "untrt", "trt"),  
  levels = c("untrt", "trt"),  
  ordered=TRUE #有序因子  
)  
group
```

```
> group <- factor(  
+   c("trt", "untrt", "trt"),  
+   levels = c("untrt", "trt")  
+ )  
> levels(group)  
[1] "untrt" "trt"  
> group <- factor(  
+   c("trt", "untrt", "trt"),  
+   levels = c("untrt", "trt"),  
+   ordered=TRUE  
+ )  
> levels(group)  
[1] "untrt" "trt"  
> group  
[1] trt    untrt trt  
Levels: untrt < trt
```

示有固

参照水
向

拼写、

R矩阵适合表达量等规则数据表

所有元素必须具有相同类型

```
mat <- matrix(1:20, nrow = 5,  
byrow = TRUE)  
dim(mat)  
mat[1, 2]  
mat[1, , drop = FALSE]  
rowSums(mat)  
colMeans(mat)
```

- 使用 [行, 列] 提取
- 提取一行或一列时用 drop = FALSE 保持二维结构
- 行名和列名属于数据含义, 读入后立即检查

R矩阵适合表达量等规则数据表

所有元素必须具有相同类型

```
mat <- matrix(1:20, nrow = 5,  
byrow = TRUE)  
dim(mat)  
mat[1, 2]  
mat[1, , drop = FALSE]  
rowSums(mat)  
colMeans(mat)
```

```
> mat <- matrix(1:20, nrow = 5, byrow = TRUE)  
> dim(mat)  
[1] 5 4  
> mat[1, 2]  
[1] 2  
> mat[1, , drop = FALSE]  
      [,1] [,2] [,3] [,4]  
[1,]    1    2    3    4  
> rowSums(mat)  
[1] 10 26 42 58 74  
> colMeans(mat)  
[1]  9 10 11 12
```

R列表保存多类型数据

复杂分析结果常用嵌套列表

```
res <- list(  
  genes = c("SOX2", "OCT4"),  
  score = c(2.1, 3.4),  
  model = lm(mpg~wt, data = mtcars)  
)  
str(res)
```

典型用途

模型结果

富集分析结果

Bioconductor 对象

ggplot 图对象

R列表保存多类型数据

复杂分析结果常用嵌套列表

```
res <- list(  
  genes = c("SOX2", "OCT4"),  
  score = c(2.1, 3.4),  
  model = lm(mpg~wt, data = mtcars)  
)  
str(res)
```

```
List of 3  
 $ genes: chr [1:2] "SOX2" "OCT4"  
 $ score: num [1:2] 2.1 3.4  
 $ model:List of 12  
 ..$ coefficients : Named num [1:2] 37.29 -5.34  
 .. ..- attr(*, "names")= chr [1:2] "(Intercept)" "wt"  
 ..$ residuals : Named num [1:32] -2.28 -0.92 -2.09 1.3 -0.2 ...  
 .. ..- attr(*, "names")= chr [1:32] "Mazda RX4" "Mazda RX4 Wag" "Datsun 710" "Hornet 4 Drive" ...  
 ..$ effects : Named num [1:32] -113.65 -29.116 -1.661 1.631 0.111 ...  
 .. ..- attr(*, "names")= chr [1:32] "(Intercept)" "wt" "" "" ...  
 ..$ rank : int 2  
 ..$ fitted.values: Named num [1:32] 23.3 21.9 24.9 20.1 18.9 ...  
 .. ..- attr(*, "names")= chr [1:32] "Mazda RX4" "Mazda RX4 Wag" "Datsun 710" "Hornet 4 Drive" ...  
 ..$ assign : int [1:2] 0 1
```

典型用途

R数据框保存等长的多类型数据

每列长度相同，每列可有不同类型

```
dat <- data.frame(  
  sample = c("S1", "S2"),  
  group = c("trt", "untrt"),  
  count = c(120L, 95L)  
)  
str(dat)
```

- 数据框与矩阵不能随意互换
- `as.matrix()` 可能把全部列转成字符型
- 统计和绘图前先确认每列的 `class`

R数据框保存等长的多类型数据

每列长度相同，每列可有不同类型

```
dat <- data.frame(  
  sample = c("S1", "S2"),  
  group = c("trt", "untrt"),  
  count = c(120L, 95L)  
)  
str(dat)
```

```
> dat <- data.frame(  
+   sample = c("S1", "S2"),  
+   group = c("trt", "untrt"),  
+   count = c(120L, 95L)  
+ )  
> str(dat)  
'data.frame':   2 obs. of  3 variables:  
 $ sample: chr   "S1" "S2"  
 $ group : chr   "trt" "untrt"  
 $ count : int  120 95  
> dat  
  sample group count  
1     S1   trt  120  
2     S2 untrt   95
```

R数据框保存等长的多类型数据

每列长度相同，每列可有不同类型

```
> dat
```

	sample	group	count
1	S1	trt	120
2	S2	untrt	95

```
> as.matrix(dat)
```

	sample	group	count
[1,]	"S1"	"trt"	"120"
[2,]	"S2"	"untrt"	" 95"

- 数据框与矩阵不能随意互换
- `as.matrix()` 可能把全部列转成字符型
- 统计和绘图前先确认每列的 `class`

R中[, [[与 \$ 的返回结果不同

列表

<code>res[1]</code>	返回子列表
<code>res[[1]]</code>	返回第一个元素
<code>res\$genes</code>	按名称返回元素

数据框

<code>dat["count"]</code>	返回数据框
<code>dat[["count"]]</code>	返回向量
<code>dat[, "count", drop=FALSE]</code>	保持数据框

函数需要矩阵或数据框时，返回类型比屏幕上看起来是否相同更重要

R中[, [[与 \$ 的返回结果不同

列表

```
> dat["count"]
count
res 1 120
res 2 95
> dat[["count"]]
res [1] 120 95
> dat[, "count", drop=FALSE]
count
1 120
2 95
> dat[, "count"]
[1] 120 95
```

数据框

```
dat["count"]      返回数据框
dat[["count"]]    返回向量
dat[, "count", drop=FALSE]
保持数据框
```

函数需要矩阵或数据框时，返回类型比屏幕上看起来是否相同更重要

R中对象类型不确定时做下检查

六个高频检查函数

```
class(x)
typeof(x)
str(x)
dim(x)
head(x)
summary(x)
```

- `str()` 同时显示对象层级、元素类型和部分取值
- `dim()` 检查行列方向, `rownames()` 与 `colnames()` 检查标识
- `summary()` 适合快速发现范围异常和缺失值

R中对象类型不确定时做下检查

六个高频检查函数

```
class(x)
typeof(x)
str(x)
dim(x)
head(x)
summary(x)
```

```
> summary(dat)
```

sample		group	count
Length :2	Length :2	Min. : 95.0	
N.unique :2	N.unique :2	1st Qu.:101.2	
N.blank :0	N.blank :0	Median :107.5	
Min.nchar:2	Min.nchar:3	Mean :107.5	
Max.nchar:2	Max.nchar:5	3rd Qu.:113.8	
		Max. :120.0	

- `summary()` 适合快速发现范围异常和缺失值

) 与

R中向量化运算和回收规则

R 会按元素计算

```
x <- 1:6
```

```
x * 2
```

```
x + c(10, 20)
```

长度不整除会产生警告

```
x + c(1, 2, 3, 4)
```

- 优先使用向量化运算，减少不必要的循环
- 长度较短的向量可能被重复使用，这称为回收
- 看到 `recycling` 警告时先检查对象长度和维度

R中向量化运算和回收规则

R 会按元素计算

```
> x <- 1:6
> x * 2
[1] 2 4 6 8 10 12
> x + c(10, 20)
[1] 11 22 13 24 15 26
>
> # 长度不整除会产生警告
> x + c(1, 2, 3, 4)
[1] 2 4 6 8 6 8
```

Warning message:

```
In x + c(1, 2, 3, 4) :
  longer object length is not a multiple of shorter object length
```

- 优先使用向量化运算，减少不必要的循环
- 长度较短的向量可能被重复使用，这称为回收
- 看到 `recycling` 警告时先检查对象长度和维度

R按行、按列或列表计算

矩阵

```
rowSums(mat)
colMeans(mat)
apply(mat, 1, mad)
apply(mat, 2, mean)
```

列表与向量

```
lapply(files, read.delim)
sapply(mat, length)
vapply(mat, length, integer(1))
```

vapply 可固定返回类型

apply() 的返回类型取决于结果，不能默认认为它返回列表

R按行、按列或列表计算

矩阵

```
> mat
```

```
      [,1] [,2] [,3] [,4]  
[1,]    1    2    3    4  
[2,]    5    6    7    8  
[3,]    9   10   11   12  
[4,]   13   14   15   16  
[5,]   17   18   19   20
```

```
> rowSums(mat)
```

```
[1] 10 26 42 58 74
```

```
> colMeans(mat)
```

```
[1]  9 10 11 12
```

```
> apply(mat, 1, mad)
```

```
[1] 1.4826 1.4826 1.4826 1.4826 1.4826
```

```
> apply(mat, 2, mean)
```

```
[1]  9 10 11 12
```

列表与向量

```
lapply(files, read.delim)
```

```
sapply(x, length)
```

```
vapply(x, length,  
integer(1))
```

vapply 可固定返回类型

取决于结果，不能默认认为它返回列表

R按行、按列或列表计算

```
> sapply(mat, length)
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
> vapply(mat, length, integer(1))
[1] 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
>
> sapply(as.data.frame(mat), length)
V1 V2 V3 V4
5 5 5 5
> vapply(as.data.frame(mat), length, integer(1))
V1 V2 V3 V4
5 5 5 5
```

列表与向量

```
lapply(files, read.delim)
sapply(x, length)
vapply(x, length,
integer(1))
```

vapply 可固定返回类型

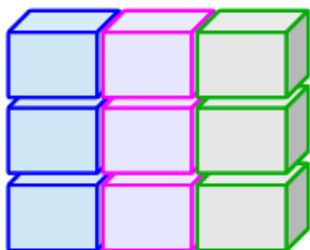
apply() 的返回类型取决于结果，不能默认认为它返回列表

R语言数据结构及其索引方式

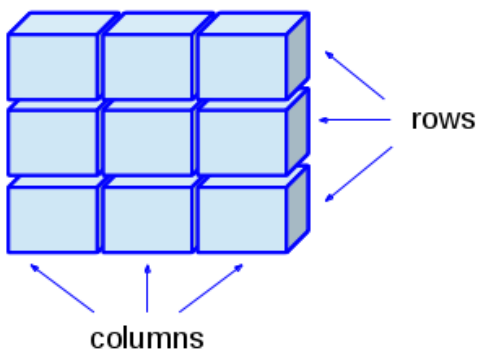
Vector



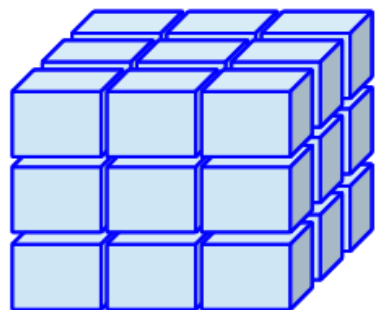
Data Frame
(Table)



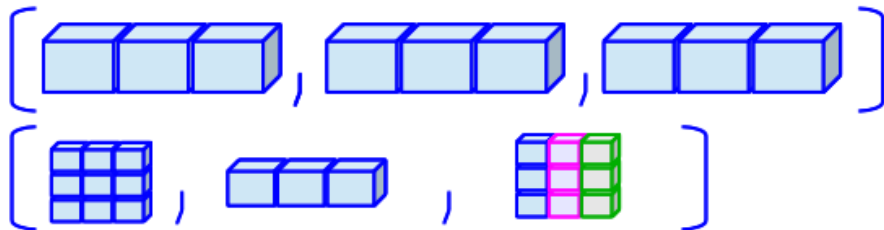
Matrix



Array



Lists



Indexing vectors

<code>x[n]</code>	nth element
<code>x[-n]</code>	all but the nth element
<code>x[1:n]</code>	first n elements
<code>x[-(1:n)]</code>	elements from n+1 to end
<code>x[c(1,4,2)]</code>	specific elements
<code>x["name"]</code>	element named "name"
<code>x[x > 3]</code>	all elements greater than 3
<code>x[x > 3 & x < 5]</code>	all elements between 3 and 5
<code>x[x %in% c("a","if")]</code>	elements in the given set

Indexing lists

<code>x[n]</code>	list with elements n
<code>x[[n]]</code>	nth element of the list
<code>x[["name"]]</code>	element named "name"
<code>x\$name</code>	as above (w. partial matching)

Indexing matrices

<code>x[i,j]</code>	element at row i, column j
<code>x[i,]</code>	row i
<code>x[,j]</code>	column j
<code>x[,c(1,3)]</code>	columns 1 and 3
<code>x["name",]</code>	row named "name"

Indexing matrices data frames (same as matrices plus the following)

<code>X[["name"]]</code>	column named "name"
<code>x\$name</code>	as above (w. partial matching)

R中文本数据读入

先明确分隔符、表头和行名

```
expr <- read.delim(  
  "data/ExpressionMatrix.txt",  
  row.names = 1,  
  check.names = FALSE  
)
```

```
meta <-  
read.delim("data/ExpressionMetadata.txt"  
)
```

- `read.delim()` 默认读取制表符文件
- 不要只看文件扩展名，先用文本编辑器检查实际分隔符
- 读入后立即运行 `dim()`、`head()` 和 `str()`

R数据读入后要做检查

- 维度：行数和列数是否符合预期
- 标识：基因 ID、样本名是否为空或重复
- 类型：数值列有没有被读成字符型
- 范围：是否存在异常极值、负值或不合理的零

检查应写进脚本

```
dim(expr)
anyDuplicated(rownames(expr))
colSums(is.na(expr))
summary(as.vector(as.matrix(expr)))
```

数据中缺失值的显示处理

NA 不能用 `x == NA` 判断

```
x <- c(2, NA, 5)
is.na(x)
mean(x)                # NA
mean(x, na.rm = TRUE)  # 3.5
complete.cases(dat)
```

- 先统计缺失值的位置和比例
- `na.rm = TRUE` 会忽略缺失值，但不会解释缺失原因
- 差异分析中过滤 `padj` 时要同时排除 NA

R中结果输出与覆盖风险

输出文件名写清对象、方向和格式

```
dir.create("results", showWarnings =
FALSE)
write.table(
  result,
  "results/DEG_trt_up.tsv",
  sep = "\t", quote = FALSE,
  row.names = FALSE
)
```

- 输入放在 data/, 结果放在 results/
- 输出前检查 nrow()、列名和前几行
- 不要把结果写回原始输入文件

数据中缺失值的显示处理

```
NA 不 > x <- c(2, NA, 5)
> is.na(x)
[1] FALSE TRUE FALSE
x <- c(2, NA, 5) > mean(x) # NA
is.na(x) [1] NA
mean(x) > mean(x, na.rm = TRUE) # 3.5
mean(x) [1] 3.5
complete.cases(dat) > complete.cases(dat)
[1] TRUE TRUE
>
> dat
  sample group count
1     S1   trt   120
2     S2 untrt   95
```

- 先统计缺失值的位置和比例
- `na.rm = TRUE` 会忽略缺失值，但不会解释缺失原因
- 差异分析中过滤 `p.adj` 时要同时排除 `NA`

R筛选、排序与保留维度

条件先保存为逻辑向量，便于检查

```
keep <- !is.na(res$padj) &  
        res$padj < 0.05 &  
        abs(res$log2FoldChange) >= 1
```

```
deg <- res[keep, , drop = FALSE]  
deg <- deg[order(deg$log2FoldChange,  
decreasing=TRUE), ]
```

- 先用 `table(keep, useNA="ifany")` 检查筛选结果
- 数据框取行时保留逗号
- 排序前确认列为 `numeric`

表达矩阵与样本信息对齐

顺序必须逐个对应

```
idx <- match(colnames(expr), meta$Sample)
stopifnot(!anyNA(idx))
meta2 <- meta[idx, , drop = FALSE]
stopifnot(identical(colnames(expr), meta2$Sample))
```

- `%in%` 只能判断是否存在，不能保证顺序一致
- `match()` 返回目标位置，可按表达矩阵列顺序重排元数据
- 绘图和建模前用 `identical()` 做最终确认

合并数据前检查键的匹配

检查键

```
anyDuplicated(id_map$ENSG)  
anyNA(id_map$ENSG)  
setdiff(rownames(res),  
id_map$ENSG)
```

执行合并

```
res$ENSG <- rownames(res)  
out <- merge(res, id_map,  
by="ENSG", all.x=TRUE,  
sort=FALSE)
```

合并后再次检查行数和重复 ID

merge() 可能改变行顺序。需要保持原顺序时，优先使用 **match()** 或显式顺序列

简单的统计：t-test两组比较

- `rnorm(10)`产生10个均值为0，标准差为1的正态分布随机数。
- `t.test` 比较两组数直接是否存在显著差异。

```
> t.test(rnorm(10),rnorm(10))
```

```
Welch Two Sample t-test
```

```
data:  rnorm(10) and rnorm(10)
t = 1.1048, df = 17.928, p-value = 0.2839
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -0.5260499  1.6922100
sample estimates:
 mean of x   mean of y 
0.52334419 -0.05973586
```

```
> t.test(rnorm(10),rnorm(10, mean=5))
```

```
Welch Two Sample t-test
```

```
data:  rnorm(10) and rnorm(10, mean = 5)
t = -13.742, df = 17.32, p-value = 9.504e-11
alternative hypothesis: true difference in means is not equal to 0
95 percent confidence interval:
 -6.630154 -4.867445
sample estimates:
 mean of x   mean of y 
-0.6582895  5.0905099
```

简单的统计：t-test两组比较

先判断独立样本还是配对样本

```
set.seed(101)
x <- rnorm(10, mean = 0)
y <- rnorm(10, mean = 1)
t.test(x, y)

# 配对设计
t.test(before, after, paired
= TRUE)
```

- 独立性来自实验设计，不由软件检验产生
- 样本很少或分布异常时同时查看原始点和稳健方法
- 报告效应大小、置信区间和P 值

简单的统计：anova多组比较

- `data(iris)` # 加载R内容测试数据——鸢尾花数据
- `head(iris)` # 显示数据格式
- # anova统计不同物种花萼长宽数据
- `model = aov(Sepal.Length ~ Species, data=iris)`
- `Tukey_HSD <- TukeyHSD(model, ordered = TRUE, conf.level = 0.95)` # TukeyHSD检验
- # 显示统计结果
- `Tukey_HSD$Species`

```
> data(iris) # 加载R内容测试数据——鸢尾花数据
> head(iris) # 显示数据格式
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4          0.2  setosa
2          4.9         3.0          1.4          0.2  setosa
3          4.7         3.2          1.3          0.2  setosa
4          4.6         3.1          1.5          0.2  setosa
5          5.0         3.6          1.4          0.2  setosa
6          5.4         3.9          1.7          0.4  setosa
> # anova统计不同物种花瓣宽度数据
> model = aov(Sepal.Length ~ Species, data=iris)
> Tukey_HSD <- TukeyHSD(model, ordered = TRUE, conf.level = 0.95)
> # 显示统计结果
> Tukey_HSD$Species
```

	diff	lwr	upr	p adj
versicolor-setosa	0.930	0.6862273	1.1737727	3.386180e-14
virginica-setosa	1.582	1.3382273	1.8257727	2.997602e-15
virginica-versicolor	0.652	0.4082273	0.8957727	8.287558e-09

简单的统计：anova多组比较

总体检验后再做多重比较

```
fit <- aov(Sepal.Length ~ Species, data = iris)
summary(fit)
TukeyHSD(fit)
```

先看数据分布

```
boxplot(Sepal.Length ~ Species, data = iris)
```

- 模型公式左侧是响应变量，右侧是分组或解释变量
- 检查残差、方差和异常值
- 重复测量、批次或协变量需要更合适的模型

组学分析中的多重假设检验

单个检验

P 值描述在零假设下观察到当前或更极端数据的概率

一个 P 值不能表示效应大小

成千上万个检验

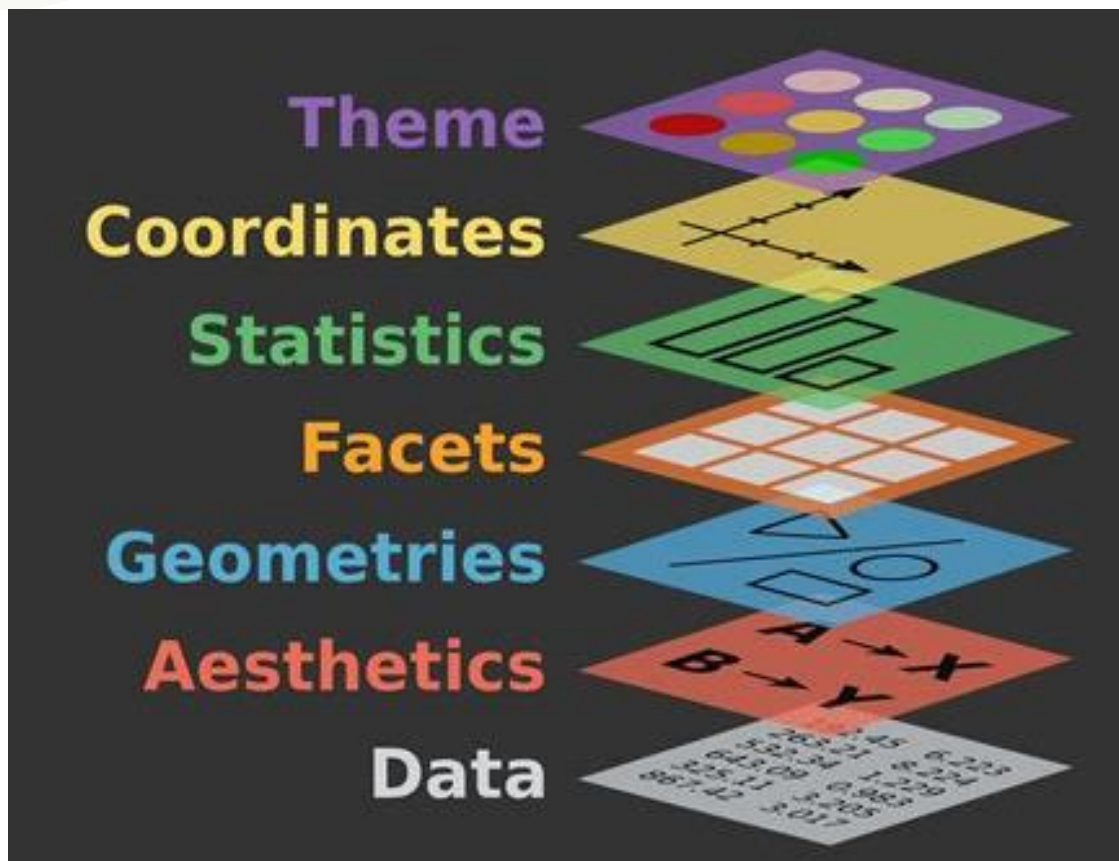
同时检验大量基因会积累假阳性

```
p.adjust(p, method =  
"BH") 控制 FDR
```

分别记录原始 P 值与校正值

```
sum(p < 0.05)  
padj <- p.adjust(p, method = "BH")  
sum(padj < 0.05, na.rm = TRUE)
```

图形语法基本概念



数据

每一行代表一个观察，每一列代表一个变量

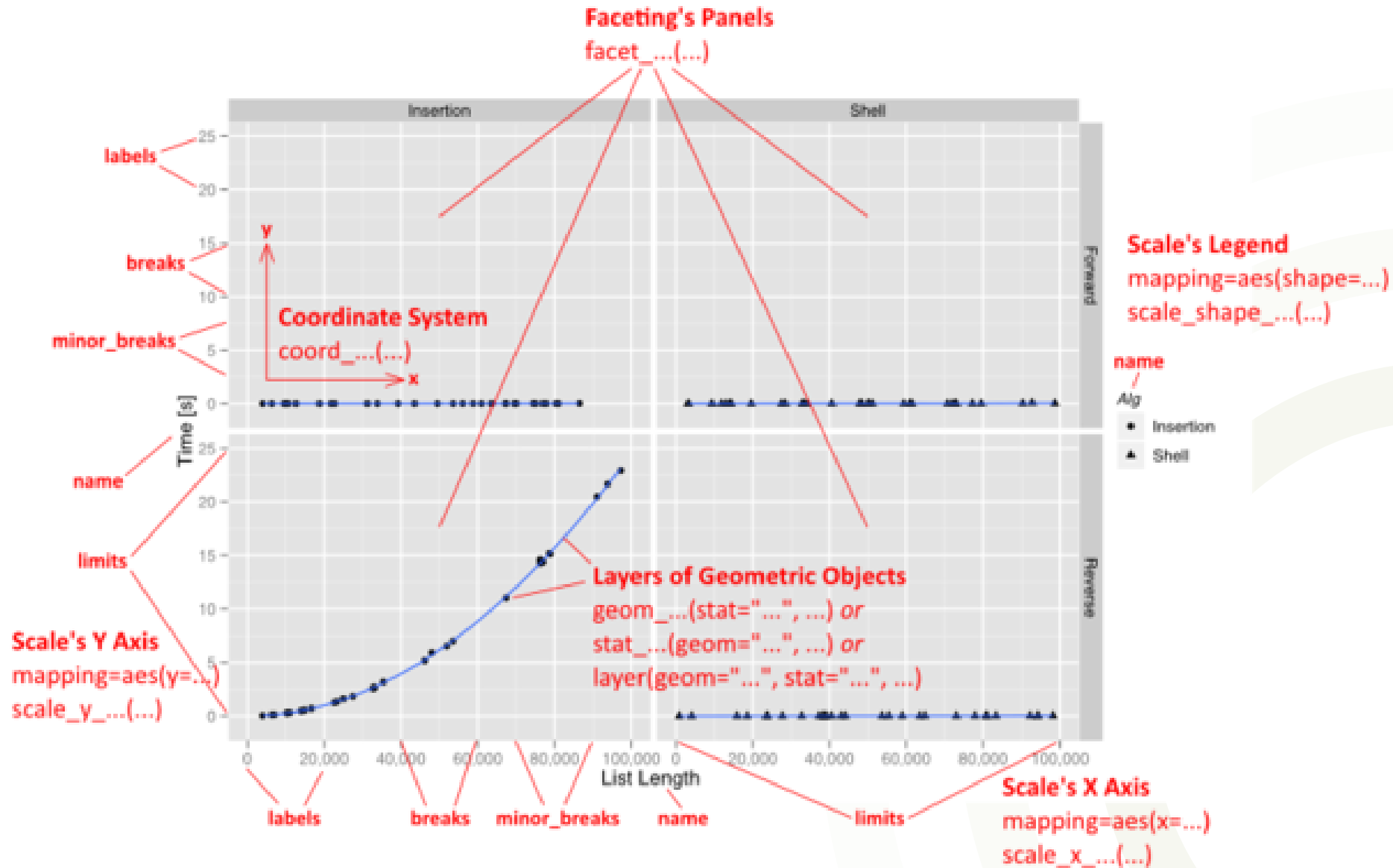
映射与几何对象

`aes()` 把变量映射到位置、颜色或大小
`geom_*()` 决定点、线、柱或箱线图

标度、坐标与主题

控制显示范围、颜色和非数据元素

图形各部分元素解释



ggplot2 用图形及其属性展示数据的特征

- 统计图就是用把数据映射到几何形状如点、线、柱的美学属性如颜色、大小、形状上
- A statistical graphic is a mapping from **data** to **aesthetic attributes** (colour, shape, size) of **geometric objects** (points, lines, bars)
- 从数据信息到像素和颜色的转换过程，在ggplot2中，被称为“标度变换” (scaling)

宽矩阵和长矩阵

表达矩阵适合计算

行：基因
列：样本
值：表达量

适合热图、聚类和矩阵运算

长表适合 ggplot2 绘图

一列 Gene
一列 Sample
一列 Expression

适合映射分组、分面和统计图

`reshape2::melt()` 可用 `tidyr` 取代

```
long <- tidyr::pivot_longer(  
  expr_df, -Gene,  
  names_to = "Sample", values_to = "Expression"  
)
```

ggplot2图绘图前准备 – 读入数据

- # 读入数据
- data <- read.table(file, header=T, row.names=NULL, sep=";")
- # 随机取出6条数据, 看下格式
- data[sample(1:nrow(data),6),]

OTU	Sample	Group	Abundance
OTU3	WT2	WT	30
OTU1	WT1	WT	2
OTU4	WT2	WT	25
OTU5	WT3	WT	10
OTU3	K03	K0	16
OTU3	K01	K0	16

文件内容

```
OTU;Sample;Group;Abundance
OTU1;K01;K0;1.2
OTU2;K01;K0;1.4
OTU3;K01;K0;16
OTU4;K01;K0;10
OTU5;K01;K0;25
OTU6;K01;K0;30
OTU7;K01;K0;2
OTU8;K01;K0;3
OTU1;K02;K0;1.2
OTU2;K02;K0;1.4
```

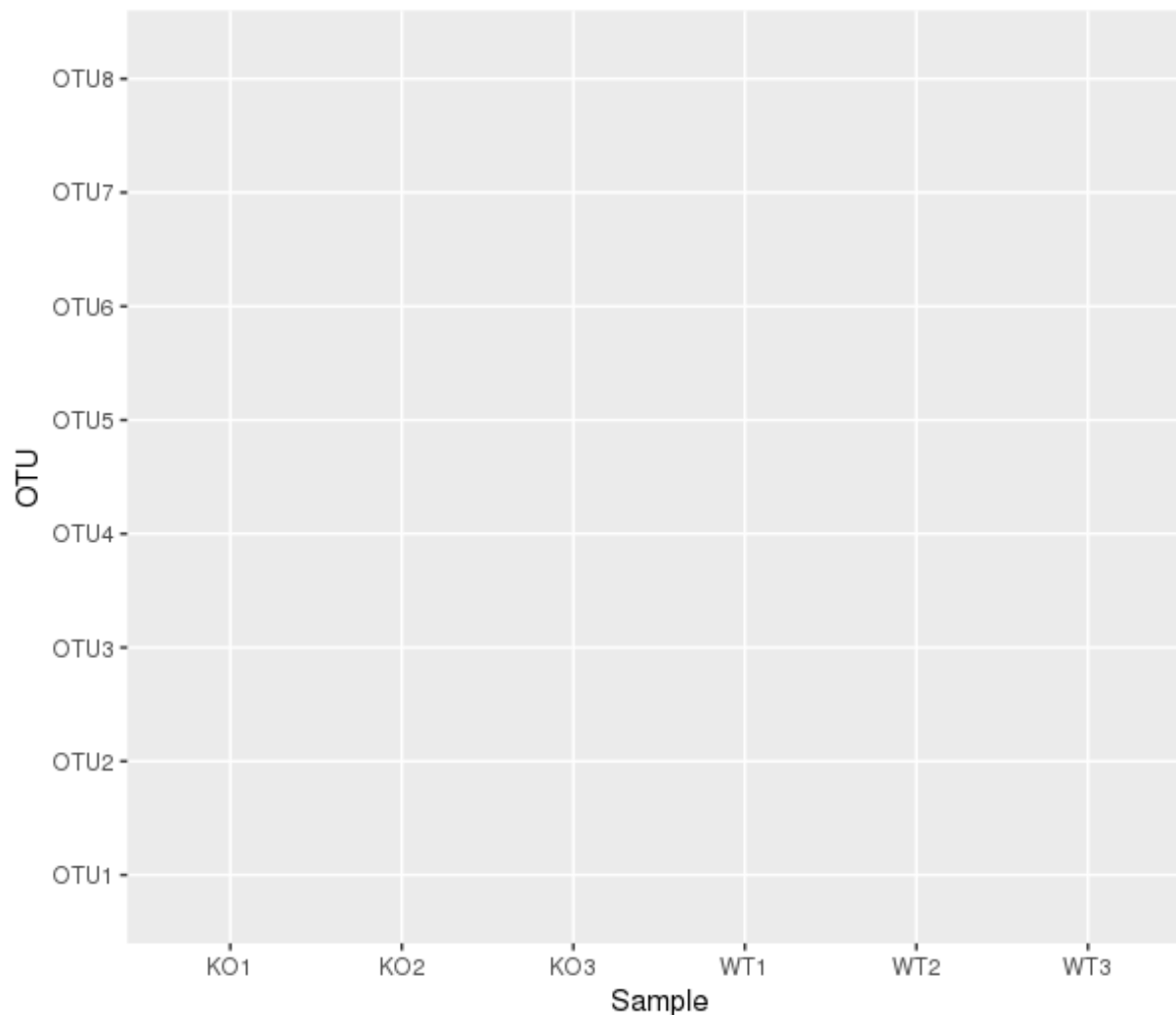
ggplot2绘图 - 初始化

```
library(ggplot2)

p <- ggplot(data,
  aes(x=Sample, y=OTU))

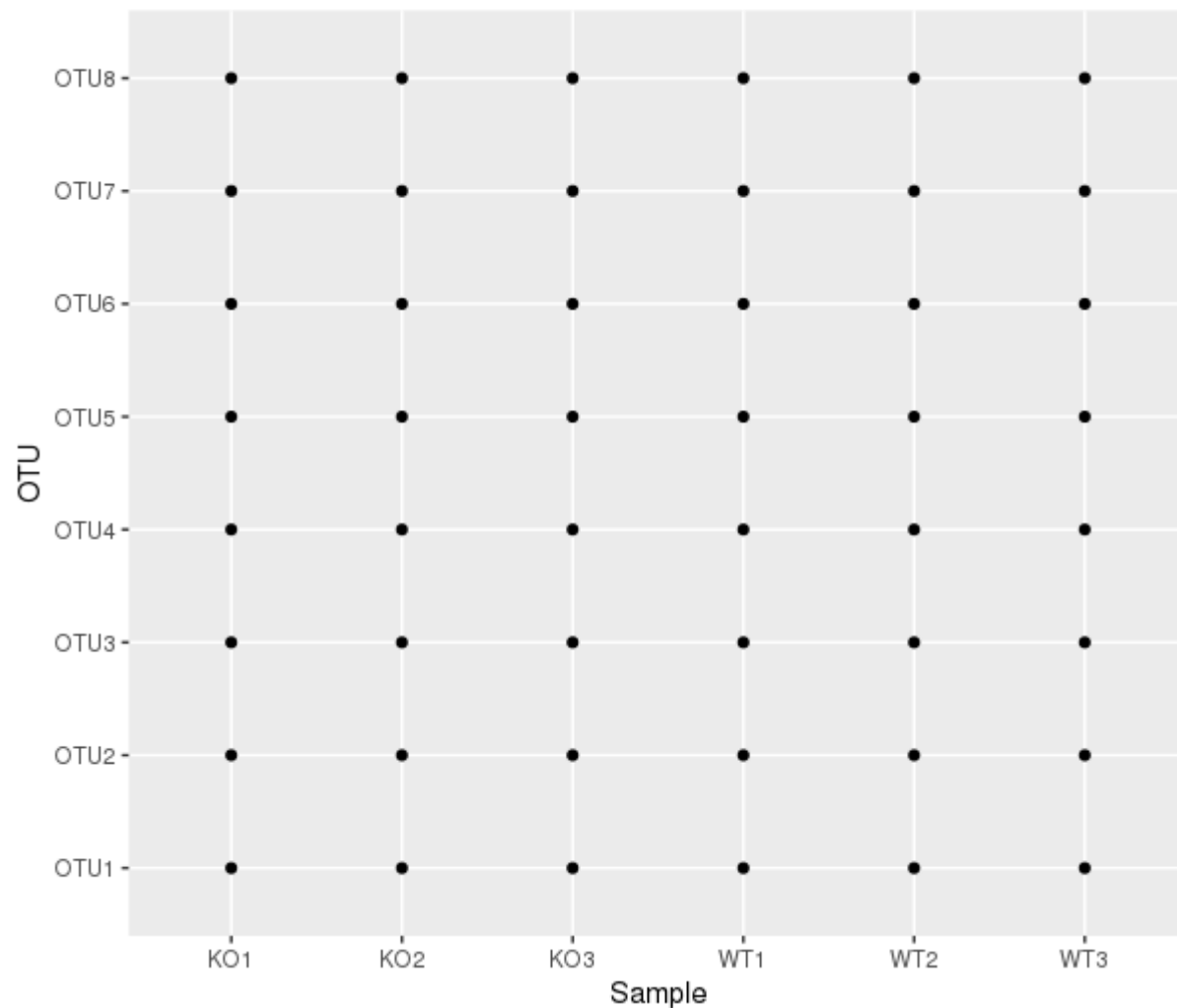
p
```

aes() 内是映射, **aes()** 外是设置



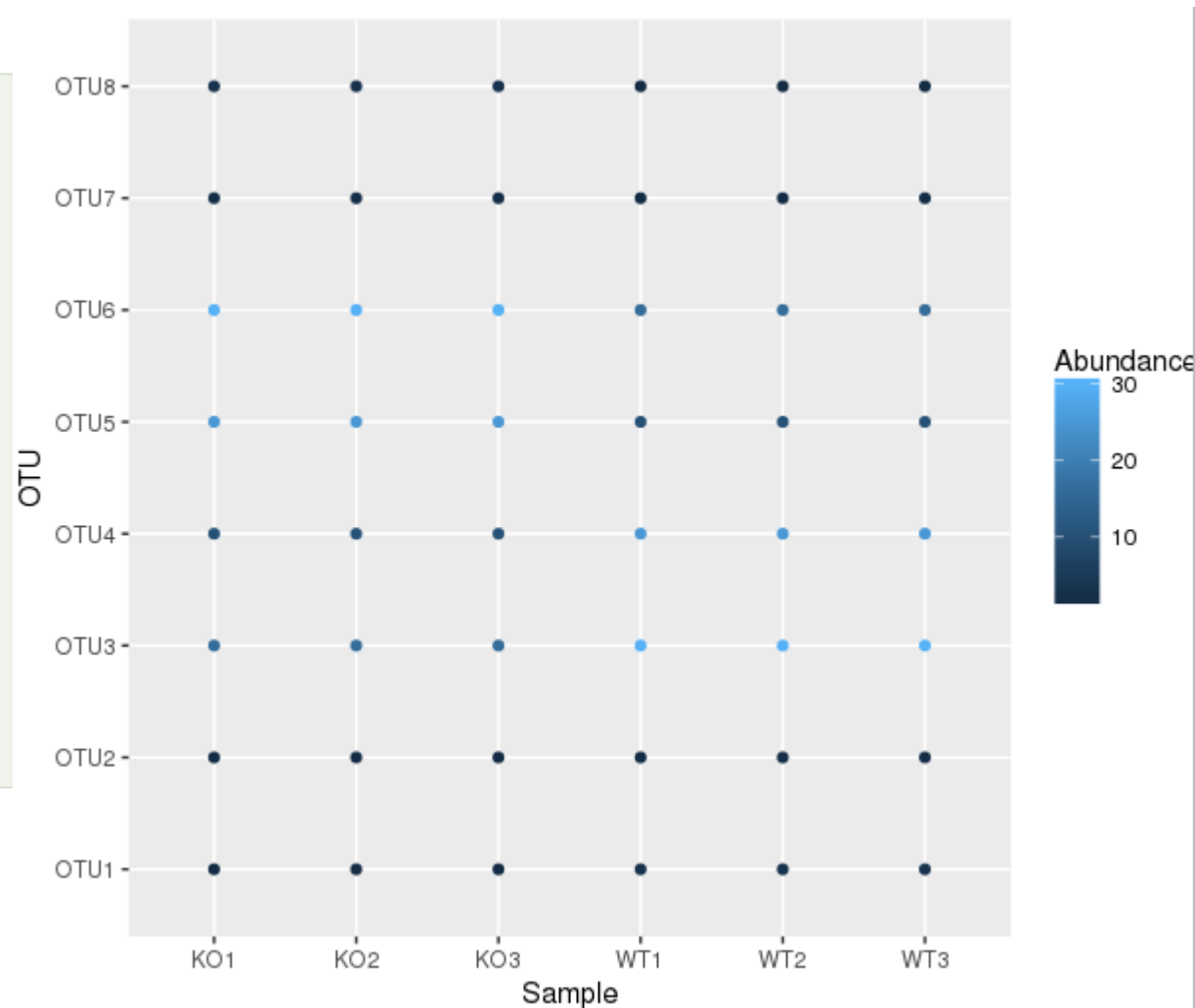
ggplot2图绘图 - 指定几何对象

```
p + geom_point()
```



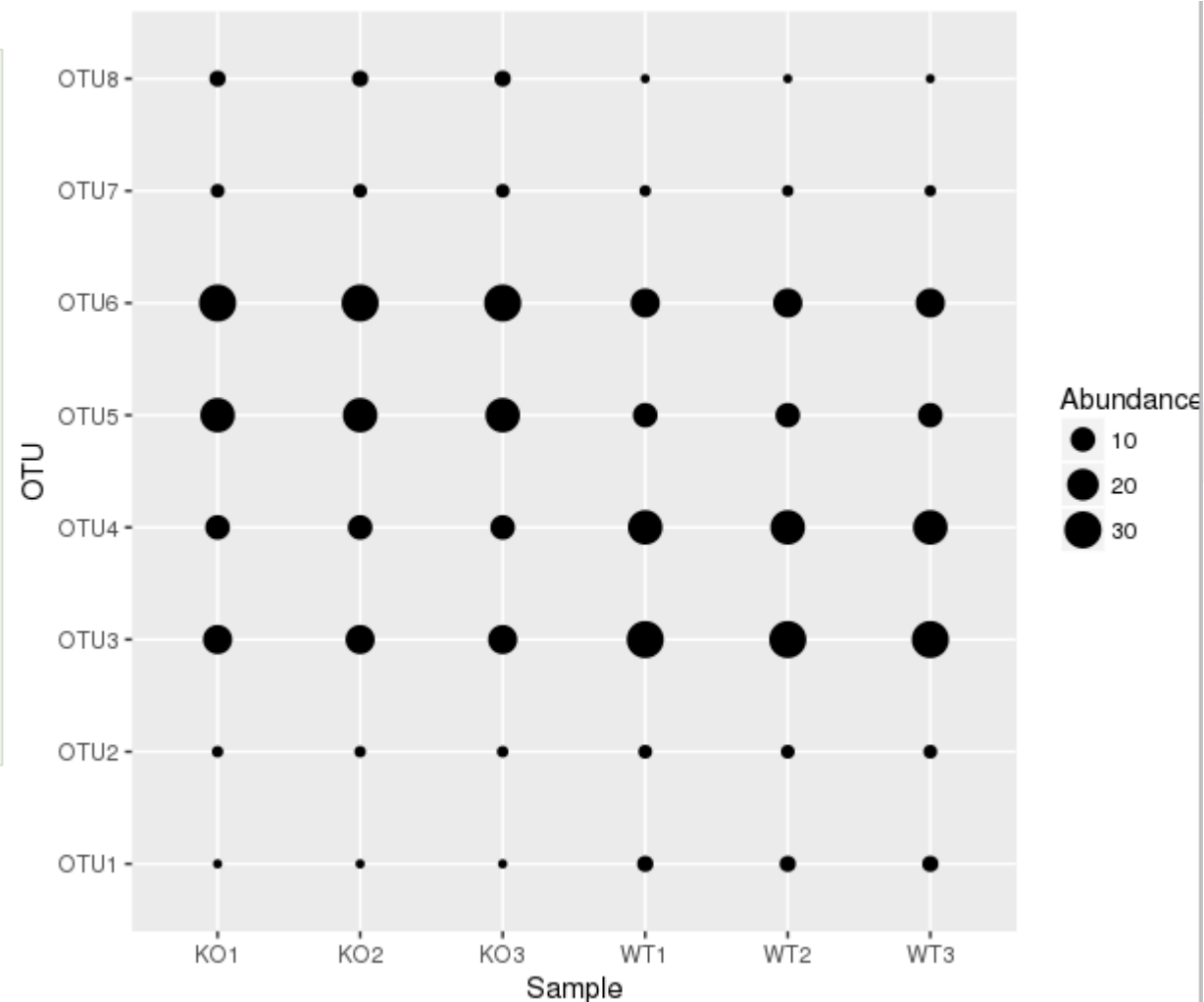
ggplot2绘图 – 映射数据到几何对象的属性

```
p +  
geom_point(aes(color=Abundance))
```



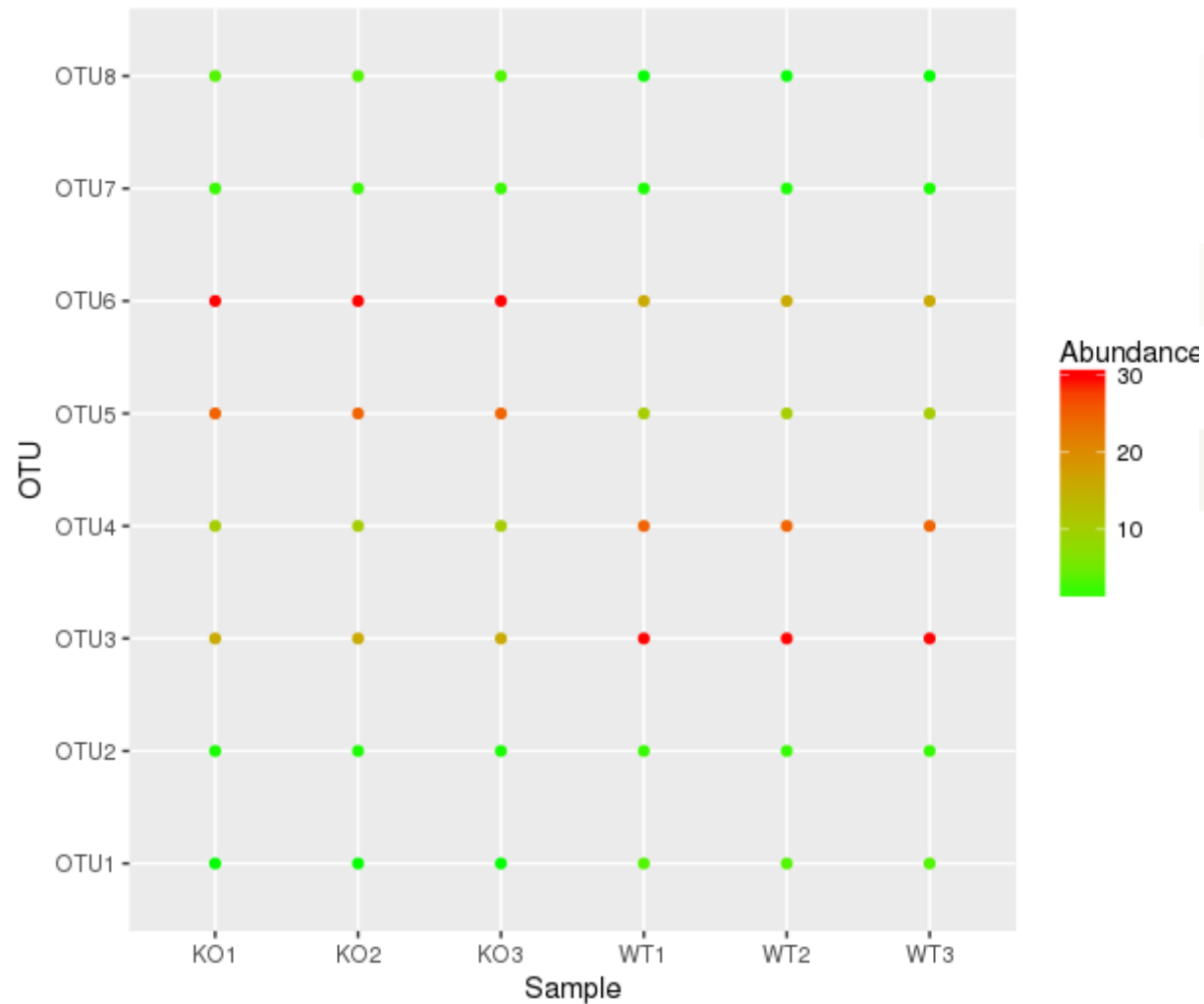
ggplot2绘图 – 映射数据到几何对象的属性

```
p +  
geom_point(aes(size=Abundance))
```



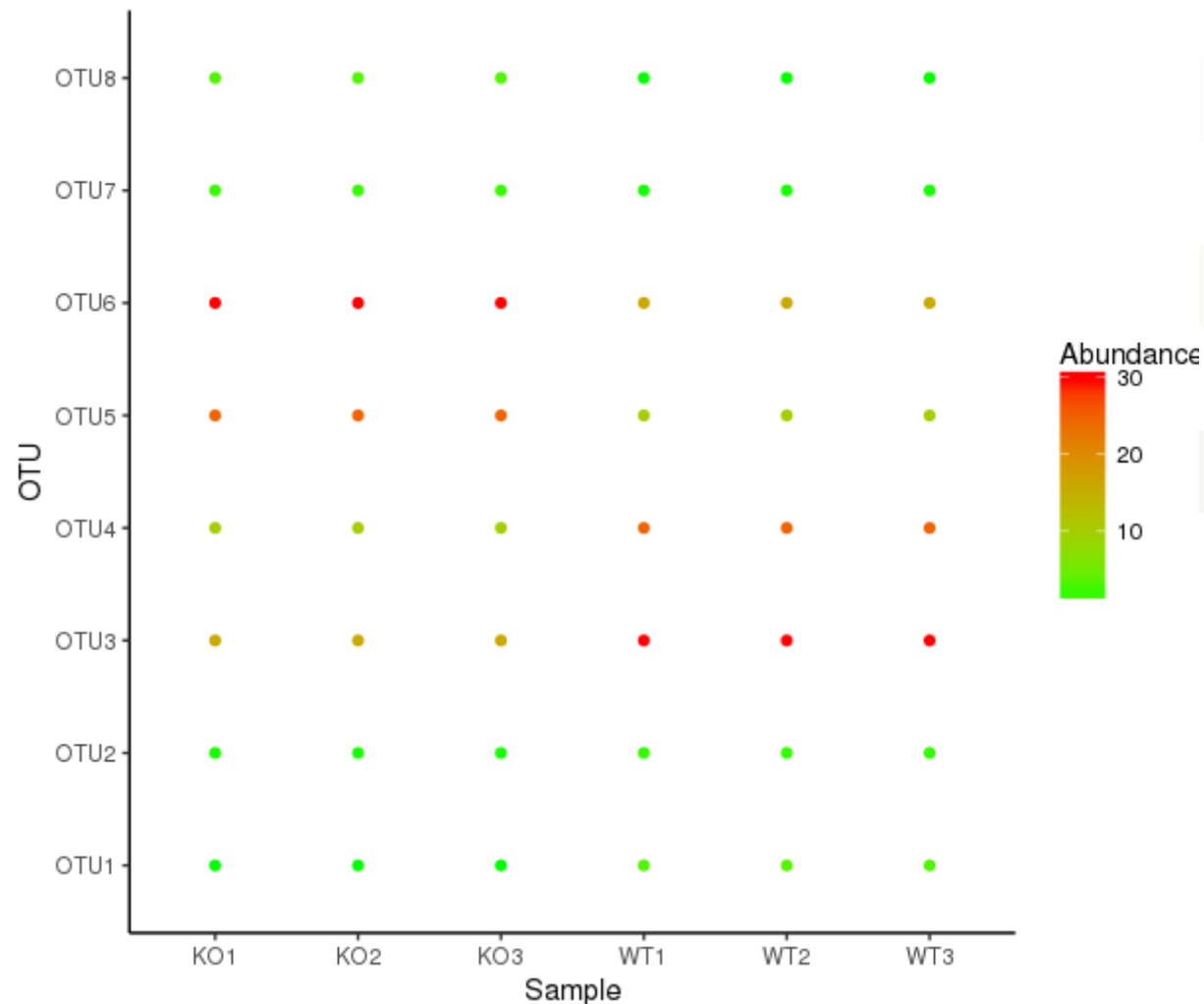
ggplot2绘图 – 调整颜色属性

```
p +  
  geom_point(aes(color=Abundance)) +  
  scale_color_continuous(low="green", high="red")
```



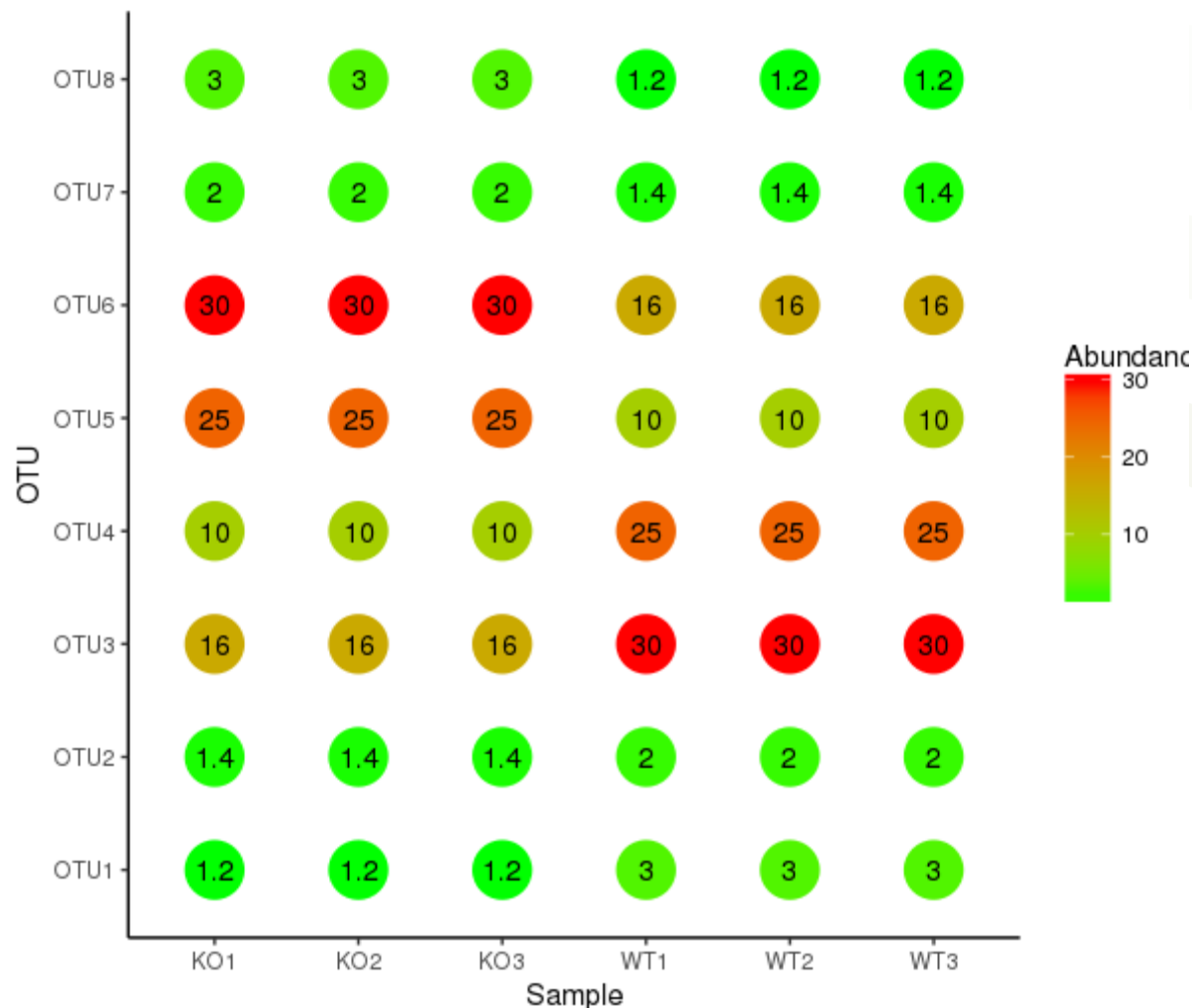
ggplot2绘图 – 更换主题

```
p +  
  geom_point(aes(color=Abundanc  
e)) +  
  scale_color_continuous(low="g  
reen", high="red") +  
  theme_classic()
```



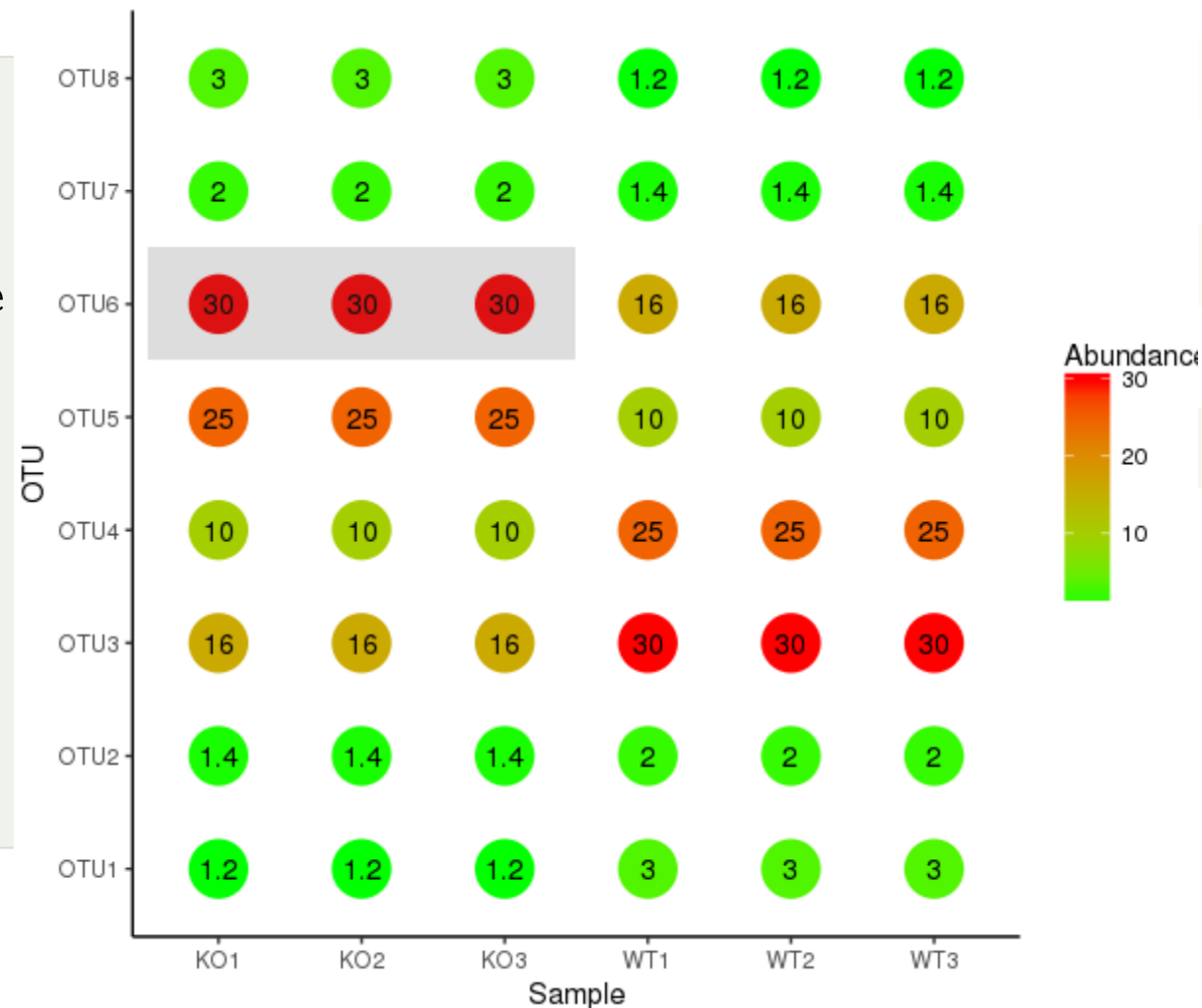
ggplot2绘图 – 放大点和填充文字

```
p +  
  geom_point(aes(color=Abundance), size=10) +  
  scale_color_continuous(low="green", high="red") +  
  geom_text(aes(label=Abundance)) +  
  theme_classic()
```



ggplot2绘图 – 增加注释

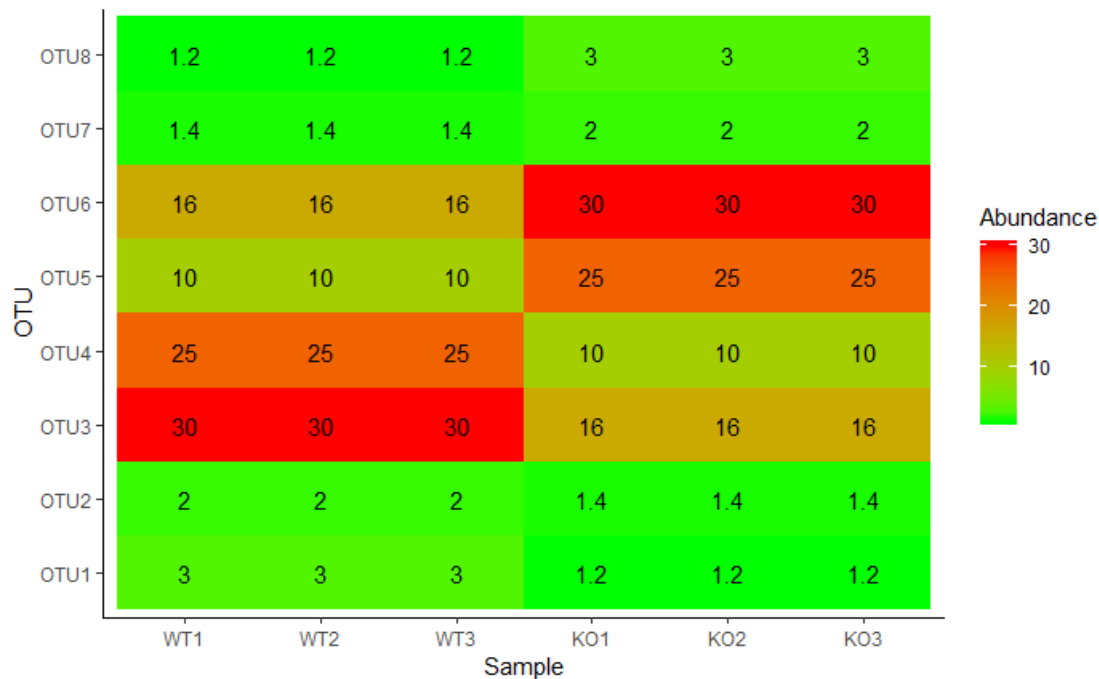
```
p +  
  geom_point(aes(color=Abundance),  
    size=10) +  
  scale_color_continuous(low="green", high="red") +  
  geom_text(aes(label=Abundance)) +  
  
  annotate("rect", xmin=0.5,  
    xmax=3.5,  
    ymin=5.5, ymax=6.5, alpha=0.2) +  
  theme_classic()
```



ggplot2绘图 – 调整横轴的顺序

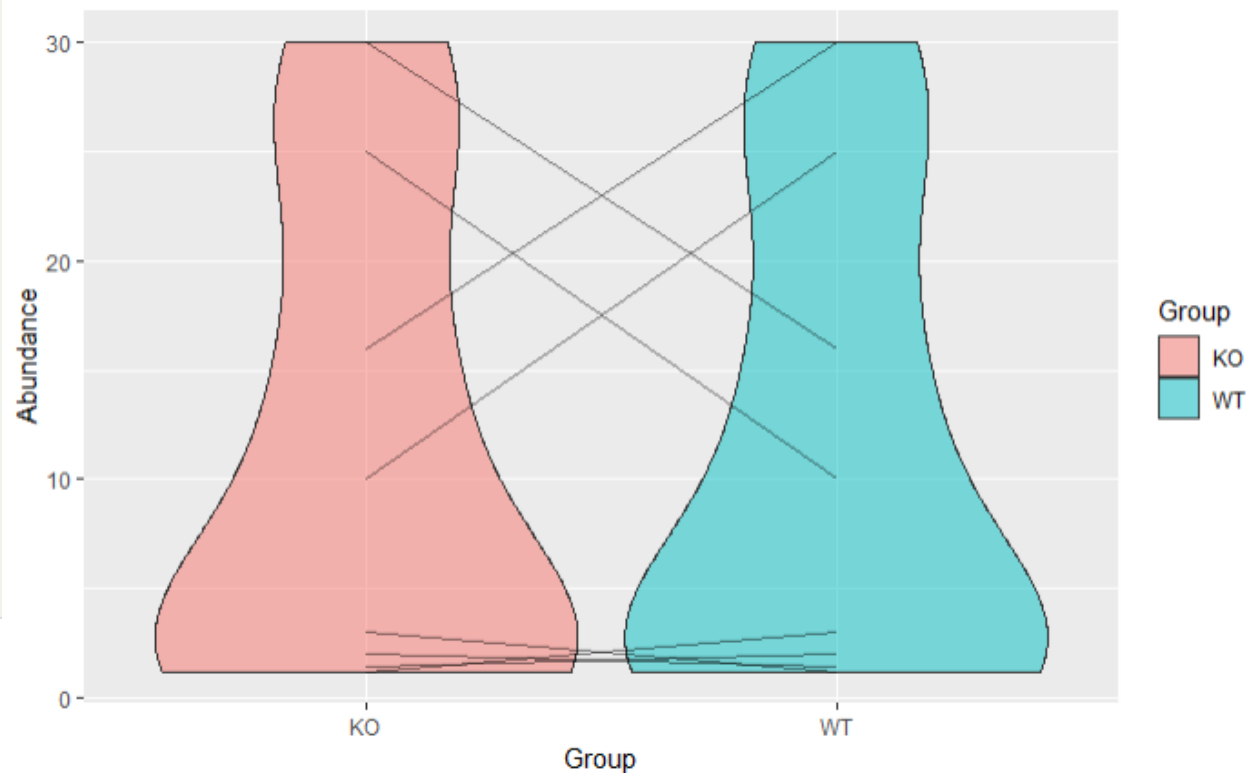
```
paste0(rep(c("WT", "KO"), each=3), rep(1:3))  
data$Sample <-  
factor(data$Sample, levels=c("WT1", "WT2", "WT3",  
"KO1", "KO2", "KO3"), ordered=T)  
p <- ggplot(data, aes(x=Sample, y=OTU)) +  
  geom_tile(aes(fill=Abundance)) +  
  scale_fill_continuous(low="green",  
high="red") +  
  geom_text(aes(label=Abundance)) +  
  theme_classic()  
p
```

先设置数据顺序，再绘图



ggplot2绘制小提琴图 – 增加连线

```
p <- ggplot(data,  
  aes(x=Group, y=Abundance)) +  
  geom_violin(aes(fill=Group),  
    alpha=0.5) +  
  geom_line(aes(group=OTU),  
    alpha=0.5)  
p
```

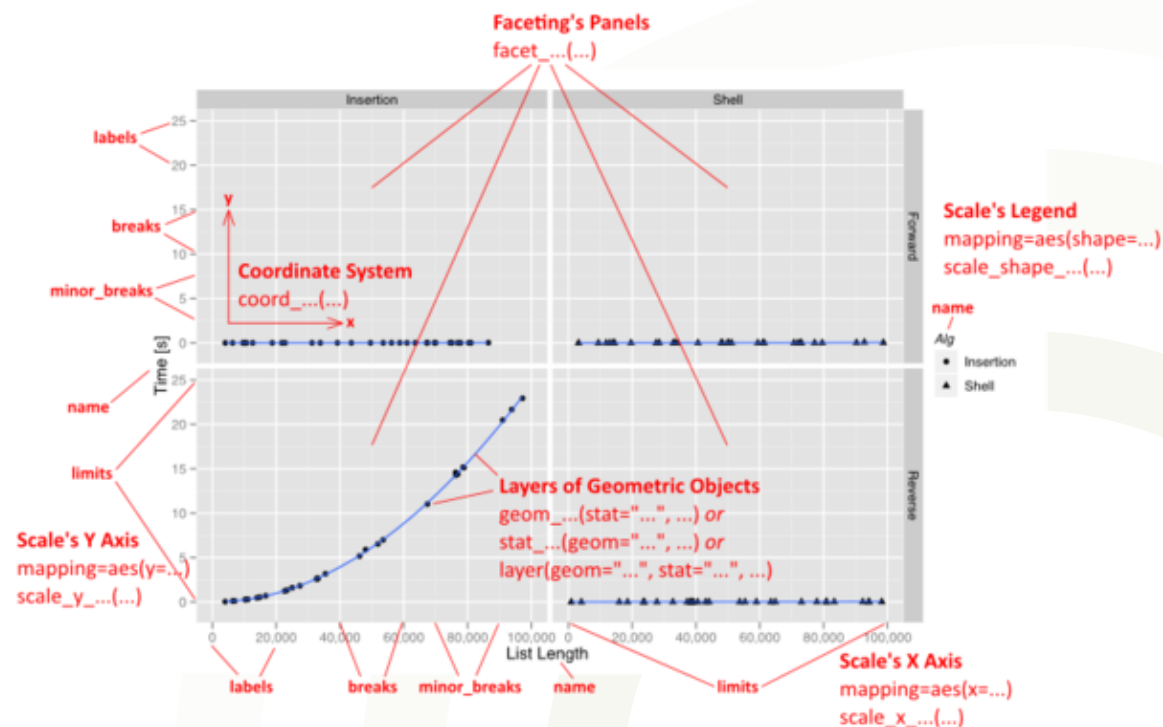


图形标注与非数据元素

标题和单位写清变量含义

```
p +  
  labs(x = "Sample", y = "Expression",  
        color = "Group") +  
  theme_classic(base_size = 14) +  
  theme(legend.position = "right")
```

- 标注应解释数据，不要遮挡点和误差线
- 坐标轴范围变化可能改变视觉判断
- 保存图形时明确宽、高和分辨率

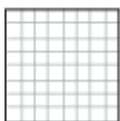


ggplot2 cheatsheet – 图形类型和变量类型

Two Variables

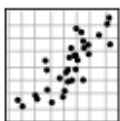
Continuous X, Continuous Y

`f <- ggplot(mpg, aes(cty, hwy))`



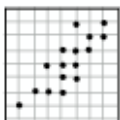
f + geom_blank()

(Useful for expanding limits)



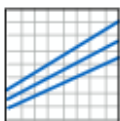
f + geom_jitter()

x, y, alpha, color, fill, shape, size



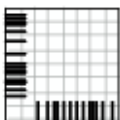
f + geom_point()

x, y, alpha, color, fill, shape, size



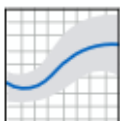
f + geom_quantile()

x, y, alpha, color, linetype, size, weight



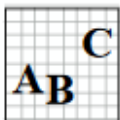
f + geom_rug(sides = "bl")

alpha, color, linetype, size



f + geom_smooth(method = lm)

x, y, alpha, color, fill, linetype, size, weight



f + geom_text(aes(label = cty))

x, y, label, alpha, angle, color, family, fontface, hjust, lineheight, size, vjust

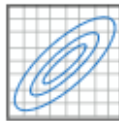
Continuous Bivariate Distribution

`i <- ggplot(movies, aes(year, rating))`



i + geom_bin2d(binwidth = c(5, 0.5))

xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size, weight



i + geom_density2d()

x, y, alpha, colour, linetype, size

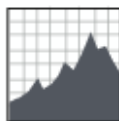


i + geom_hex()

x, y, alpha, colour, fill size

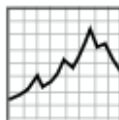
Continuous Function

`j <- ggplot(economics, aes(date, unemploy))`



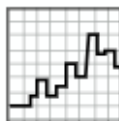
j + geom_area()

x, y, alpha, color, fill, linetype, size



j + geom_line()

x, y, alpha, color, linetype, size



j + geom_step(direction = "hv")

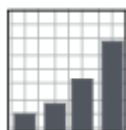
x, y, alpha, color, linetype, size

Visualizing error

ggplot2 cheatsheet – 图形类型和变量类型

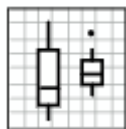
Discrete X, Continuous Y

`g <- ggplot(mpg, aes(class, hwy))`



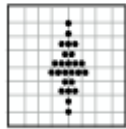
`g + geom_bar(stat = "identity")`

x, y, alpha, color, fill, linetype, size, weight



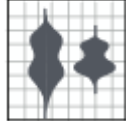
`g + geom_boxplot()`

lower, middle, upper, x, ymax, ymin, alpha, color, fill, linetype, shape, size, weight



`g + geom_dotplot(binaxis = "y", stackdir = "center")`

x, y, alpha, color, fill

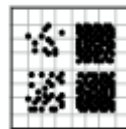


`g + geom_violin(scale = "area")`

x, y, alpha, color, fill, linetype, size, weight

Discrete X, Discrete Y

`h <- ggplot(diamonds, aes(cut, color))`



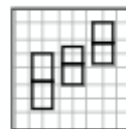
`h + geom_jitter()`

x, y, alpha, color, fill, shape, size

Visualizing error

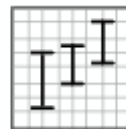
`df <- data.frame(grp = c("A", "B"), fit = 4:5, se = 1:2)`

`k <- ggplot(df, aes(grp, fit, ymin = fit-se, ymax = fit+se))`



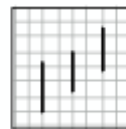
`k + geom_crossbar(fatten = 2)`

x, y, ymax, ymin, alpha, color, fill, linetype, size



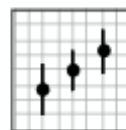
`k + geom_errorbar()`

x, ymax, ymin, alpha, color, linetype, size, width (also `geom_errorbarh()`)



`k + geom_linerange()`

x, ymin, ymax, alpha, color, linetype, size



`k + geom_pointrange()`

x, y, ymin, ymax, alpha, color, fill, linetype, shape, size

Maps

`data <- data.frame(murder = USArrests$Murder, state = tolower(rownames(USArrests)))`

`map <- map_data("state")`

`l <- ggplot(data, aes(fill = murder))`



`l + geom_map(aes(map_id = state), map = map) +`

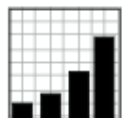
`expand_limits(x = map$long, y = map$lat)`

map_id, alpha, color, fill, linetype, size

ggplot2 cheatsheet – 坐标系统和属性

Coordinate Systems

`r <- b + geom_bar()`



`r + coord_cartesian(xlim = c(0, 5))`

xlim, ylim

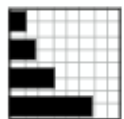
The default cartesian coordinate system



`r + coord_fixed(ratio = 1/2)`

ratio, xlim, ylim

Cartesian coordinates with fixed aspect ratio between x and y units



`r + coord_flip()`

xlim, ylim

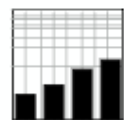
Flipped Cartesian coordinates



`r + coord_polar(theta = "x", direction=1)`

theta, start, direction

Polar coordinates



`r + coord_trans(ytrans = "sqrt")`

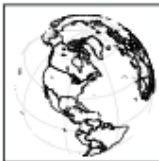
xtrans, ytrans, limx, limy

Transformed cartesian coordinates. Set xtrans and ytrans to the name of a window function.

`z + coord_map(projection = "ortho", orientation=c(41, -74, 0))`

projection, orientation, xlim, ylim

Map projections from the mapproj package (mercator (default), azequalarea, lagrange, etc.)



Color and fill scales

Discrete



`n <- b + geom_bar(aes(fill = fl))`



`n + scale_fill_brewer(palette = "Blues")`
For palette choices:
library(RColorBrewer)
display.brewer.all()



`n + scale_fill_grey(start = 0.2, end = 0.8, na.value = "red")`

Continuous



`o <- a + geom_dotplot(aes(fill = ..x..))`



`o + scale_fill_gradient(low = "red", high = "yellow")`



`o + scale_fill_gradient2(low = "red", high = "blue", mid = "white", midpoint = 25)`



`o + scale_fill_gradientn(colours = terrain.colors(6))`
Also: rainbow(), heat.colors(), topo.colors(), cm.colors(), RColorBrewer::brewer.pal()

Shape scales

Manual shape values



`p <- f + geom_point(aes(shape = fl))`



`p + scale_shape(solid = FALSE)`



`p + scale_shape_manual(values = c(3:7))`
Shape values shown in chart on right

0	□	6	▽	12	⊞	18	◆	24	▲
1	○	7	⊠	13	⊠	19	●	25	▼
2	△	8	✱	14	⊠	20	●	26	★
3	+	9	◇	15	■	21	●	27	●
4	×	10	⊕	16	●	22	■	28	○
5	◇	11	⊠	17	▲	23	◆	29	○

Size scales

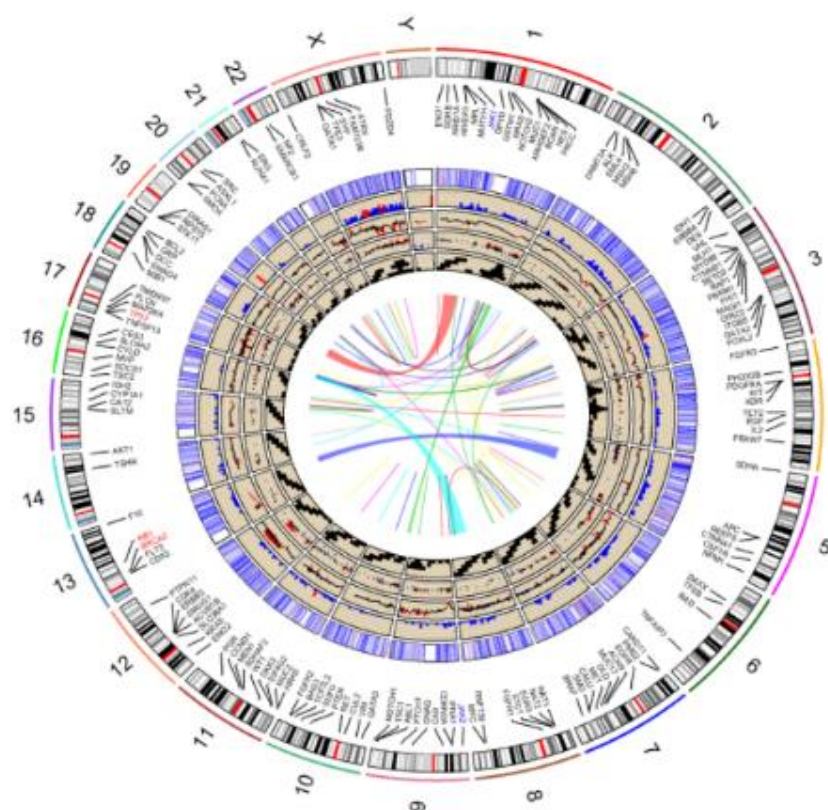
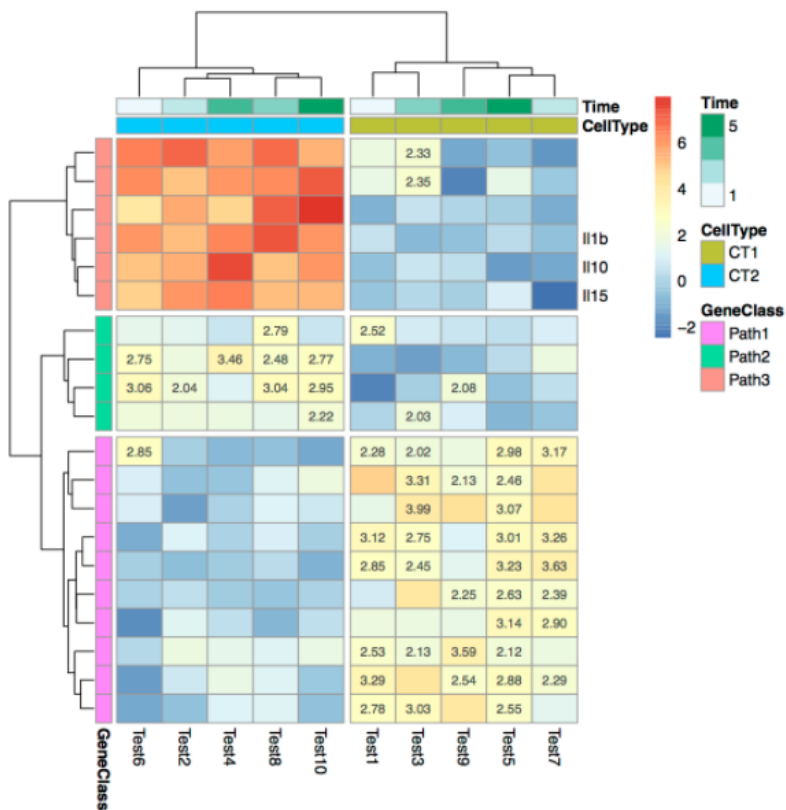


`q <- f + geom_point(aes(size = cyl))`



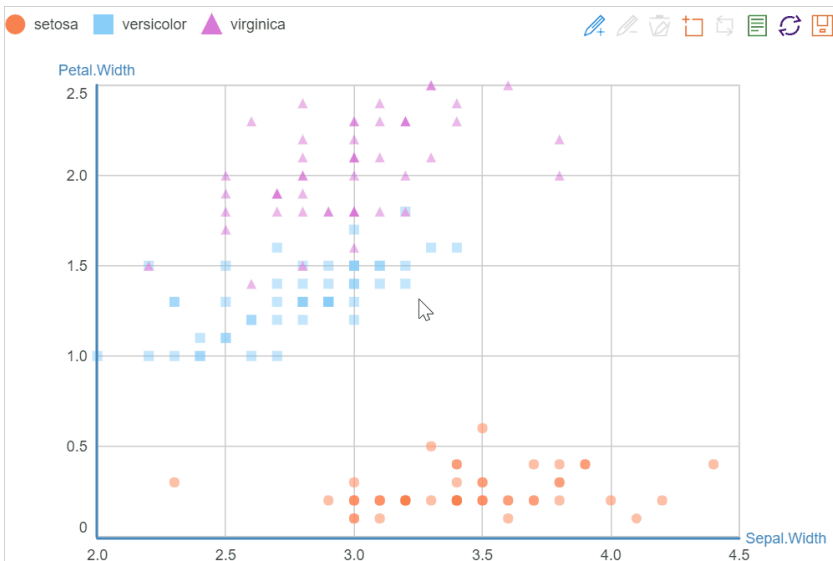
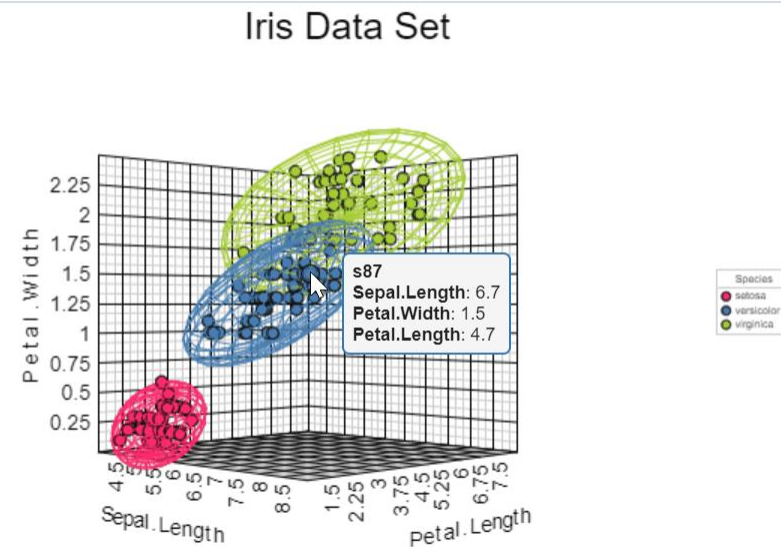
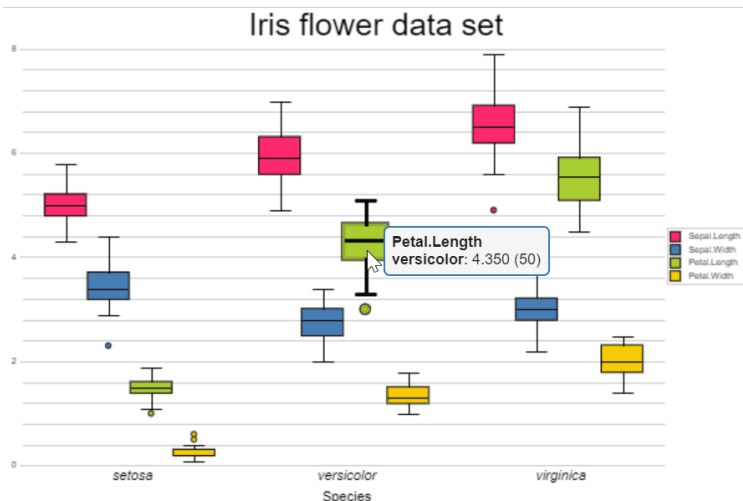
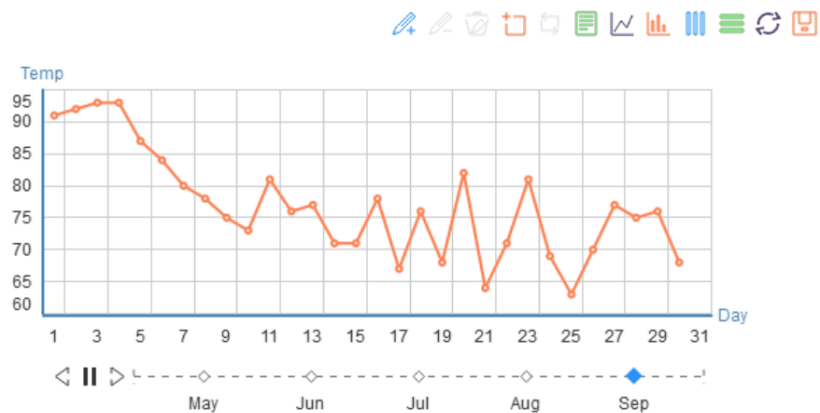
`q + scale_size_area(max = 6)`
Value mapped to area of circle (not radius)

其它专业绘图包



heatmap热图简化 http://mp.weixin.qq.com/s/_9LKs6t6rcjzokF_0gneSA
用circlize包绘制circos-plot <https://mp.weixin.qq.com/s/-bRF7OXaj091Ob1dwprMWQ>

交互式绘图



视频教程：R语言recharts包绘制交互式图形

<http://mp.weixin.qq.com/s/XOZD4ftYLPZjM2IVLYgUpA>

R语言交互式可视化包CanvasXpress <http://mp.weixin.qq.com/s/EQ7-T66YIAYvqa33Y8z7YA>

机器学习——随机森林



<https://mp.weixin.qq.com/s/PmTKKvrGhEL7oXLV9nJ7fw>

Rbasic.Rmd 的课堂运行方式

- 这是一个连续脚本，从安装检查开始逐段运行
- 每段代码运行前先读注释，确认输入对象和预期输出
- 每完成一步就检查 `class`、`dim`、`head` 和关键行数
- 报错时停在当前代码块，不跳到依赖后续对象的代码

```
getwd()  
list.files()  
R.version.string  
sessionInfo()
```

练习一：差异分析结果表

正负方向取决于 DESeq2 contrast 的定义

```
res <- read.delim(file, row.names = 1,  
check.names = FALSE)  
keep <- !is.na(res$padj) & res$padj < 0.05 &  
        abs(res$log2FoldChange) >= 1  
deg <- res[keep, , drop = FALSE]  
  
# 先确认 contrast 方向, 再命名上调结果  
table(sign(deg$log2FoldChange))
```

- 总基因数是否包含表头
- 所谓“最高表达”要明确对应哪一列
- 上调与下调文件名必须和 contrast 方向一致

练习二：基因 ID 映射

映射前检查 ID 列是否唯一

```
idx <- match(rownames(res), id_map$ENSG)
symbol <- id_map$Symbol[idx]
entrez <- id_map$Entrez[idx]

res_symbol <- res[!is.na(symbol), , drop = FALSE]
rownames(res_symbol) <-
make.unique(symbol[!is.na(symbol)])
```

- 分别统计未匹配、重复和一对多映射
- 不要在筛选后继续使用筛选前的行名向量
- 输出后检查行数、唯一 ID 和列顺序

练习三：表达矩阵与样本信息

缺失基因和样本顺序都要检查

```
genes2 <- intersect(target_genes, rownames(expr))  
mat2 <- expr[genes2, , drop = FALSE]  
meta2 <- meta[match(colnames(mat2), meta$Sample), , drop=FALSE]  
stopifnot(identical(colnames(mat2), meta2$Sample))
```

- 热图：说明是否按行标准化
- 配对图：线条按 Individual 分组
- 时间序列：先按 MAD 排序，再保留二维矩阵

AI 辅助 R 编程

适合交给 AI

解释报错信息和陌生函数

把长代码改写为可读步骤

根据表头生成筛选或绘图草稿

提出检查点和边界情况

必须人工核对

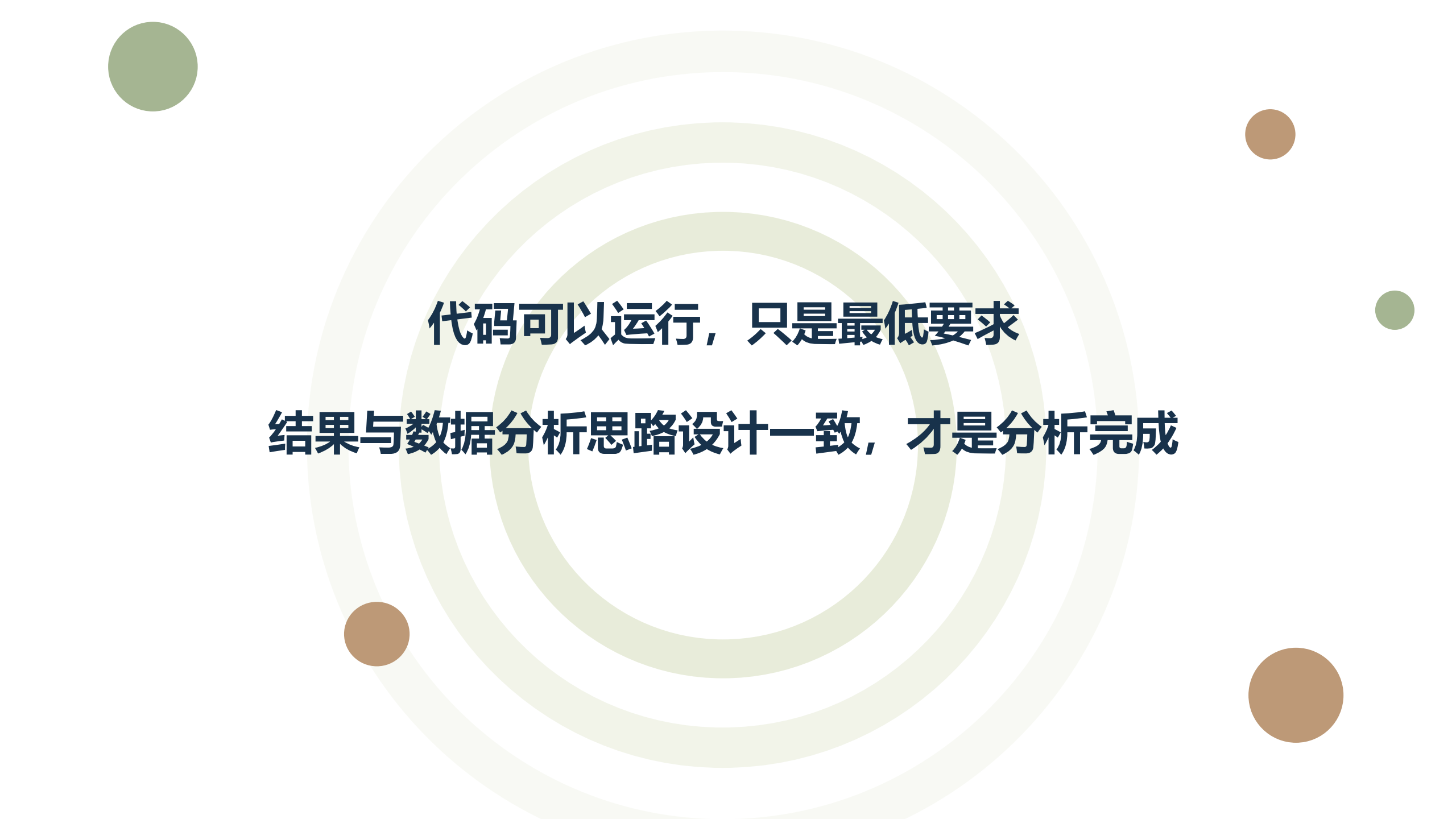
对象类型、维度和列名

统计设计与分组关系

差异方向、阈值和缺失值

命令是否覆盖或修改原文件

把 `str()`、`head()`、列名和完整报错一起提供，比只说“代码不能运行”更有效

The background features a series of concentric circles in a light olive green color, centered on the slide. Scattered around these circles are several solid-colored dots in shades of olive green and brown. The text is centered within the innermost circle.

代码可以运行，只是最低要求
结果与数据分析思路设计一致，才是分析完成